

Du pipeline au superscalaire

M2 Secil : KINN9HU
Pascal SAINRAT

Limite du pipeline

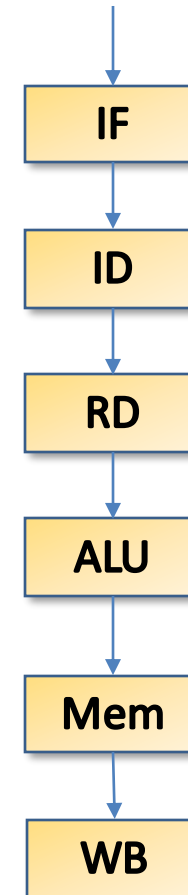
- Bande passante maximum : une instruction par cycle
- Les différents types d'instruction n'ont pas tout à fait le même traitement
 - La latence d'une instruction est plus longue que nécessaire
- Le pipeline est rigide : le gel d'un étage bloque tous les étages antérieurs

Tt des instructions dans un pipeline générique

42%	25%	12%	21%
40%	25%	15%	20%
Calcul	Load	Store	Branch ^t
Accès I\$	Accès I\$	Accès I\$	Accès I\$
Décodage	Décodage	Décodage	Décodage
Lecture regs	Lecture regs	Lecture regs	Lecture regs
Calcul	Calcul @	Calcul @	Calcul @
	Lect. Mem.	Ecr. Mem.	
Ecr. Reg.	Ecr. Reg.		Ecr. PC.

[Jourdan Sainrat, 95]

[Agerwala cocke, 87]

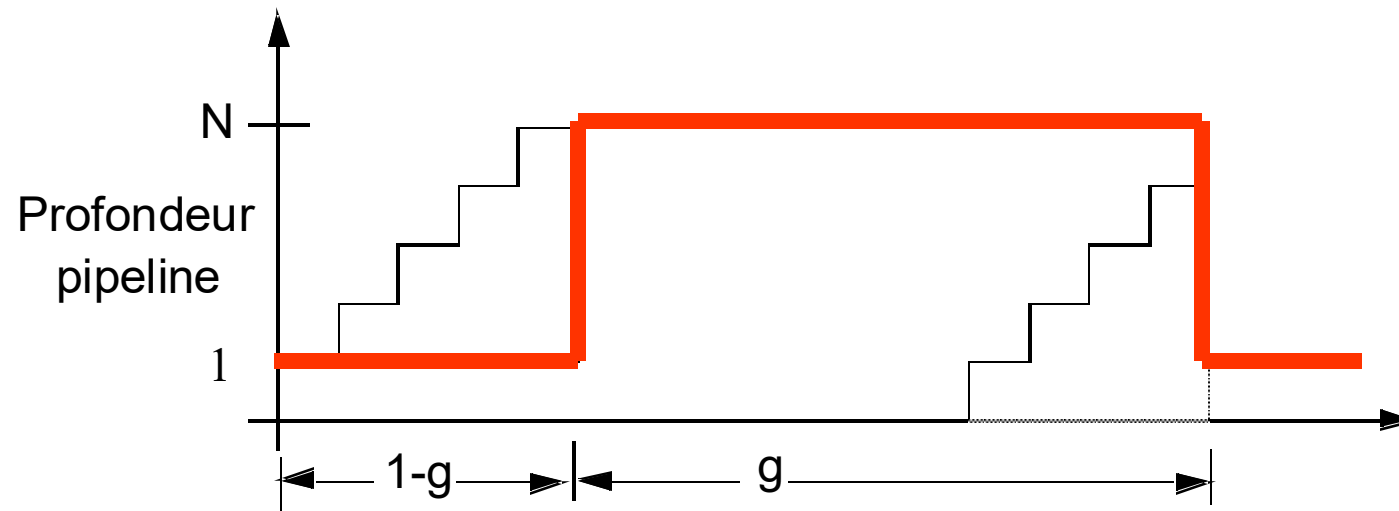


Bande passante du pipeline

$$Performance = \frac{1}{NB\ instructions} \times \frac{instructions}{cycle} \times \frac{1}{temps\ de\ cycle} = \frac{IPC \times fréquence}{Nombre\ d'instructions}$$

- La fréquence peut être augmentée en employant un pipeline plus profond mais le gain sur la fréquence peut être contrebalancé par l'overhead du pipeline, les pénalités dues aux dépendances.
- Pipeline scalaire = une instruction par étage. IPCmax limité à un.
- Pour dépasser cette limite, plus d'une instruction par étage : pipeline parallèle.
- 80's CPI : 5 $\Rightarrow$ 1,15 (décennie du pipeline)
- 90's CPI : 1,15 $\Rightarrow$ 0,5 (décennie du superscalaire)
- 00's multi-cœurs, parallélisme de threads

Modèle de performance du pipeline



- g : fraction du temps où le pipeline est plein
- $1-g$: fraction du temps où le pipeline n'est pas plein
- L'impact de $1-g$ est très fort. Le gel du pipeline doit être minimisé.

L'idée du superscalaire

- Loi d'Amdahl

$$Amélioration = \frac{1}{1 - f + \frac{f}{v}}$$

- Alléger le verrou du séquentiel
- Loi d'Amdahl révisée

$$Amélioration = \frac{1}{\frac{(1-f)}{s} + \frac{f}{v}}$$

Parallélisation de
la partie exécutée
séquentiellement
sur un pipeline
scalaire

Autre
parallélisation
(SIMD, multicore,
...)

Limites du parallélisme d'instructions

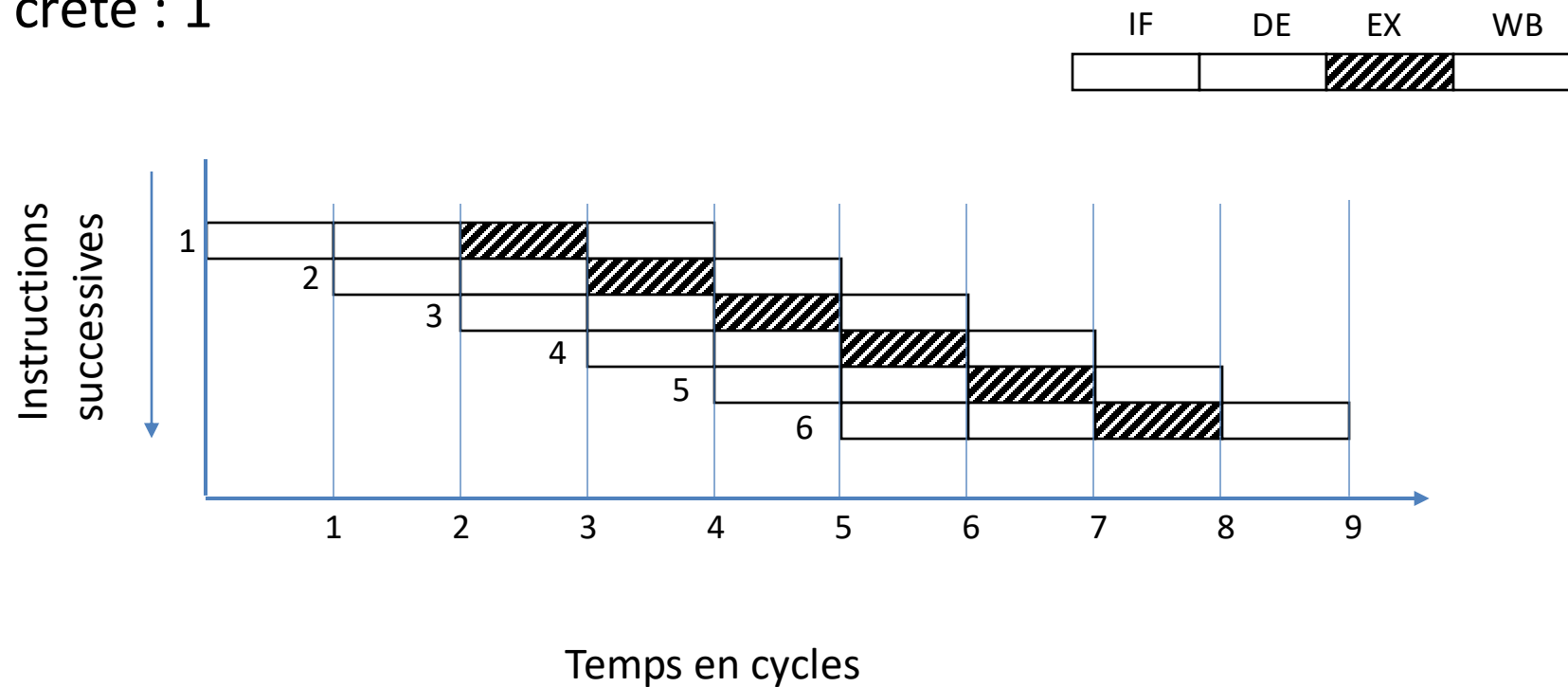
Weiss and Smith [1984]	1.58
Sohi and Vajapeyam [1987]	1.81
Tjaden and Flynn [1970]	1.86 (Flynn's bottleneck)
Tjaden and Flynn [1973]	1.96
Uht [1986]	2.00
Smith et al. [1989]	2.00
Jouppi and Wall [1988]	2.40
Johnson [1991]	2.50
Acosta et al. [1986]	2.79
Wedig [1982]	3.00
Butler et al. [1991]	5.8
Melvin and Patt [1991]	6
Wall [1991]	7 (Jouppi disagreed)
Kuck et al. [1972]	8
Riseman and Foster [1972]	51 (no control dependences)
Nicolau and Fisher [1984]	90 (Fisher's optimism)

Superscalaire

- Aller plus loin que le pipeline scalaire pour un IPC > 1
- Lancer plusieurs instructions par cycle
- Forme assez générale de parallélisme
- Conçu pour le code séquentiel qui est difficile à paralléliser autrement.
- Exploitation du parallélisme d'instruction (grain fin)

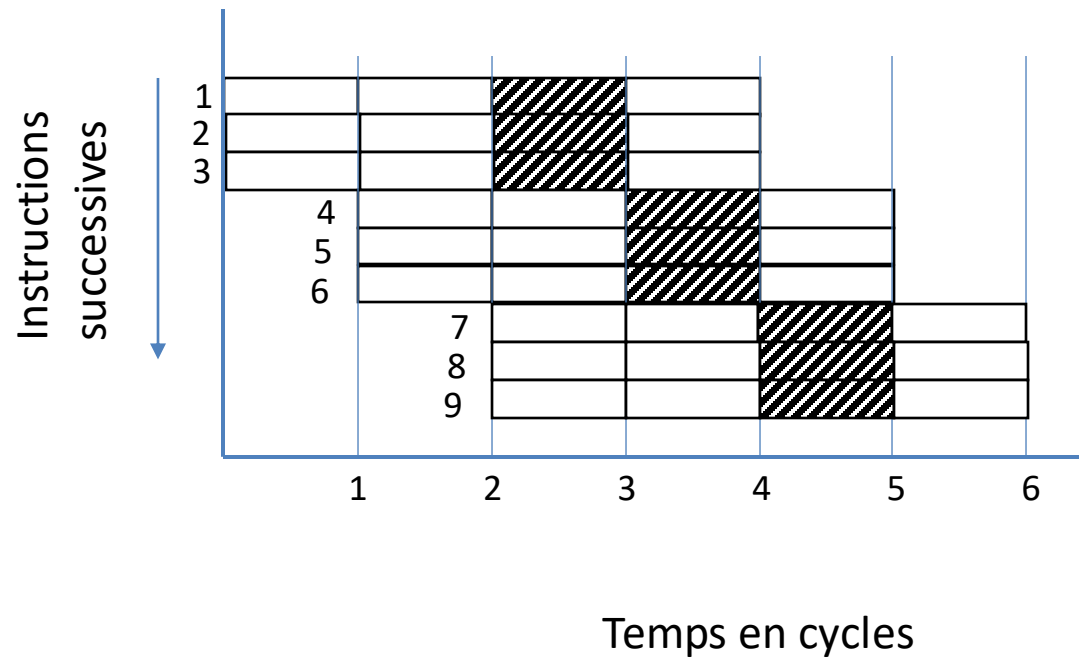
RISC scalaire de base

- Lancement d'une instruction par cycle
- Latence d'une opération : 1
- IPC crête : 1



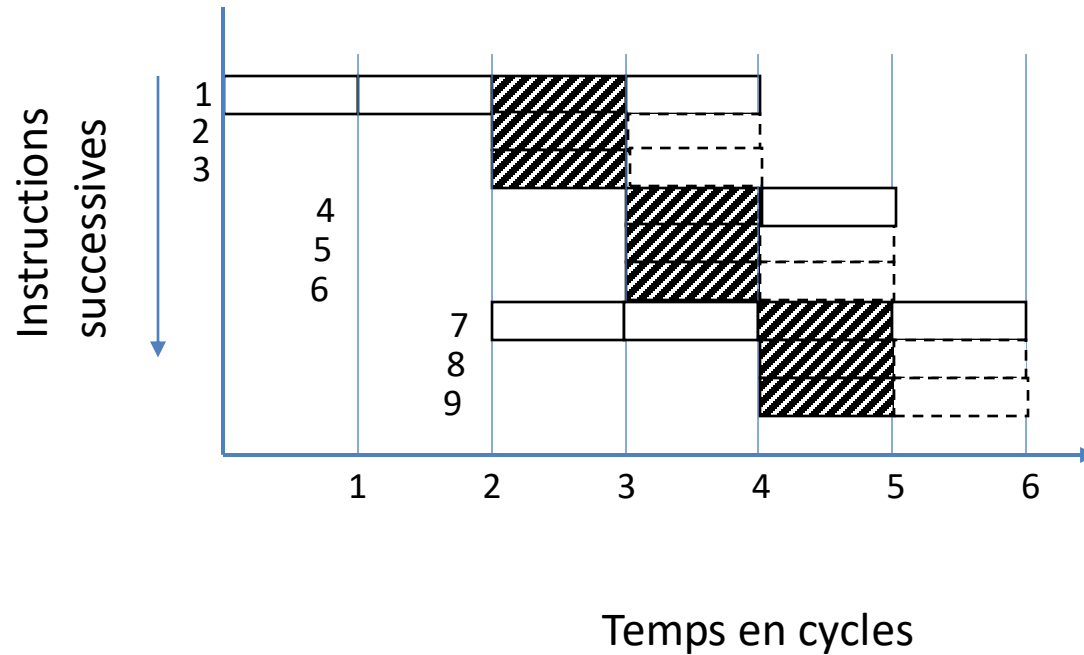
Superscalaire

- Lancement de n instructions par cycle
- Latence d'une opération : 1
- IPC crête : n

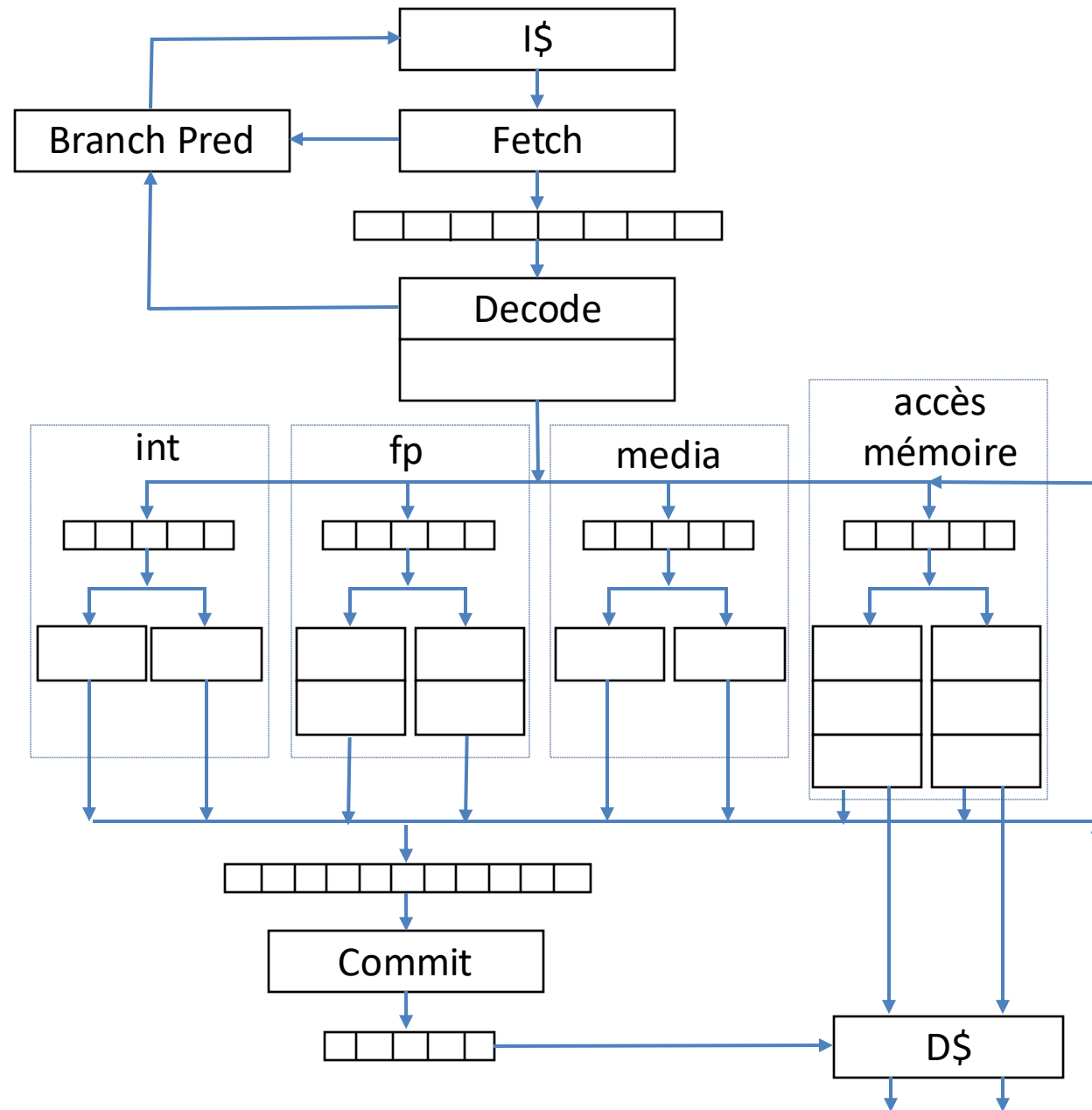


VLIW

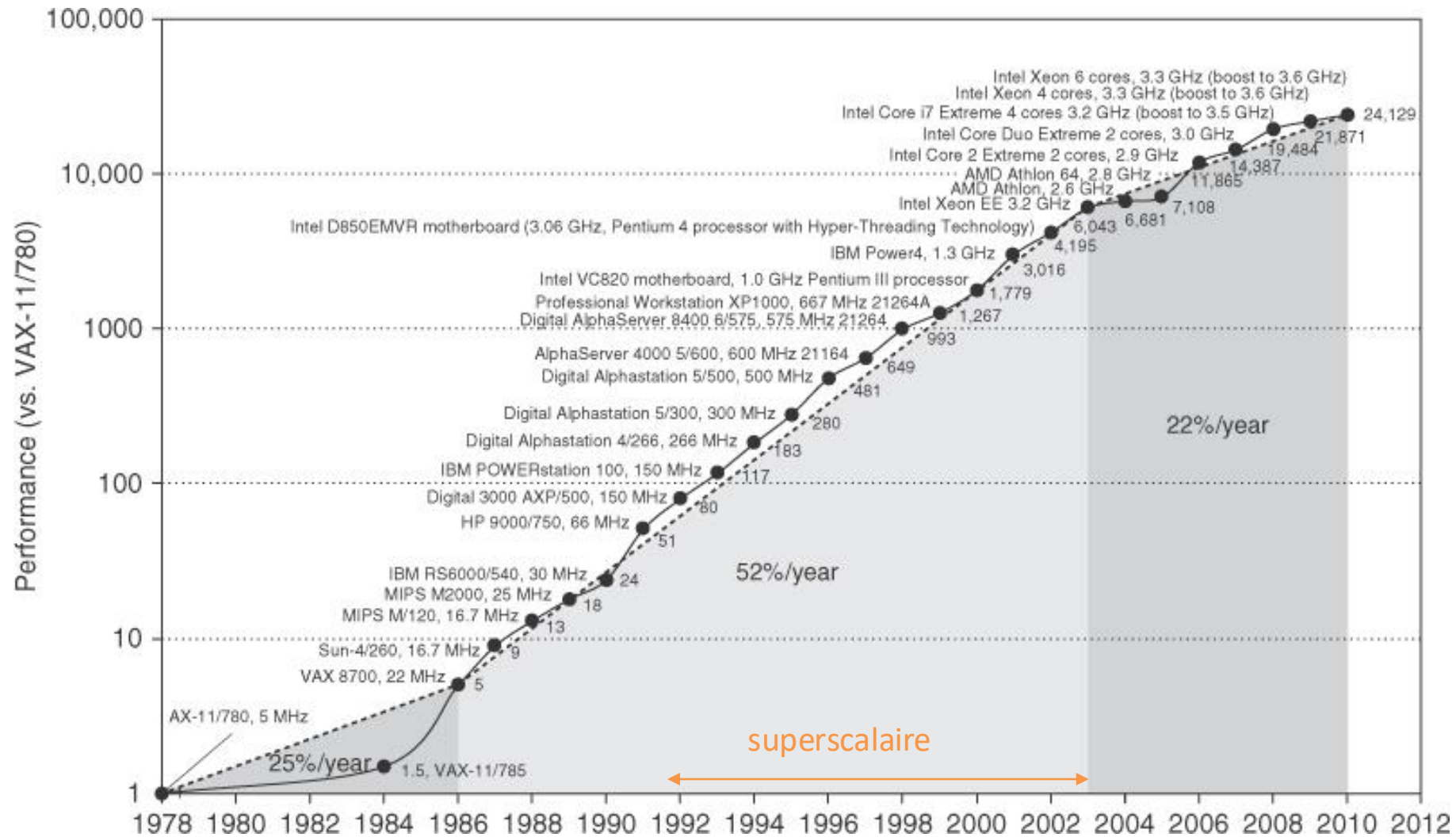
- Lancement de n instructions par cycle
- Latence d'une opération : 1
- IPC crête : n



Challenges superscalaires

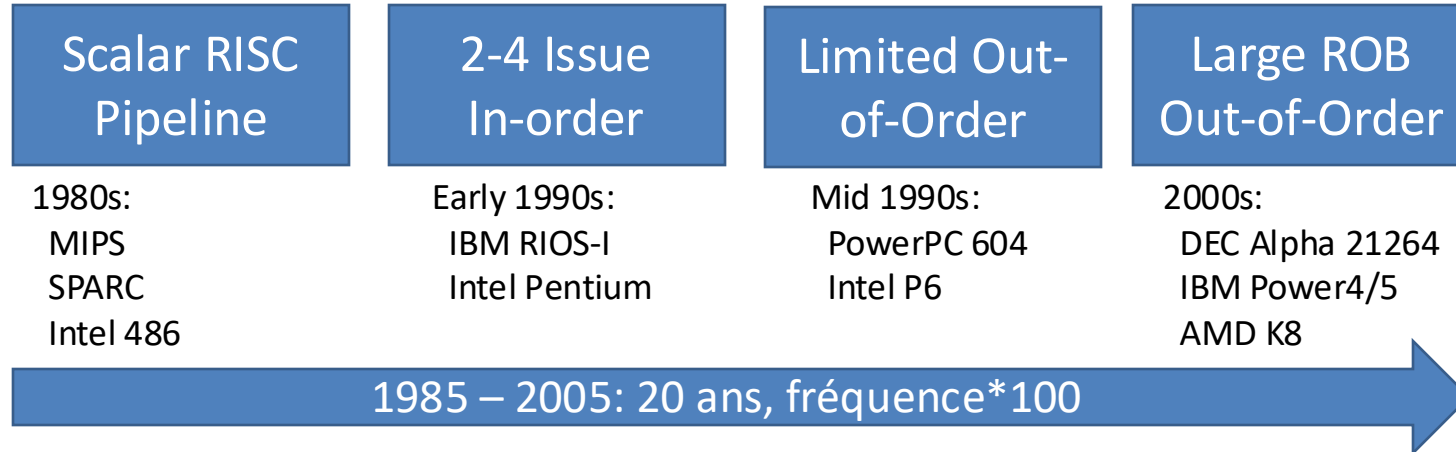


Dépasser 1 instruction/cycle

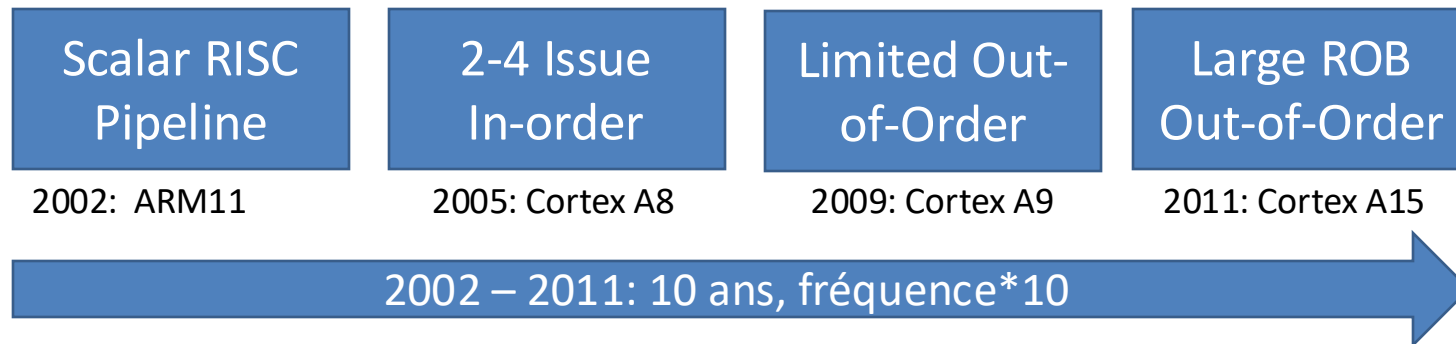


Evolution processeurs : parallélisme d'instruction

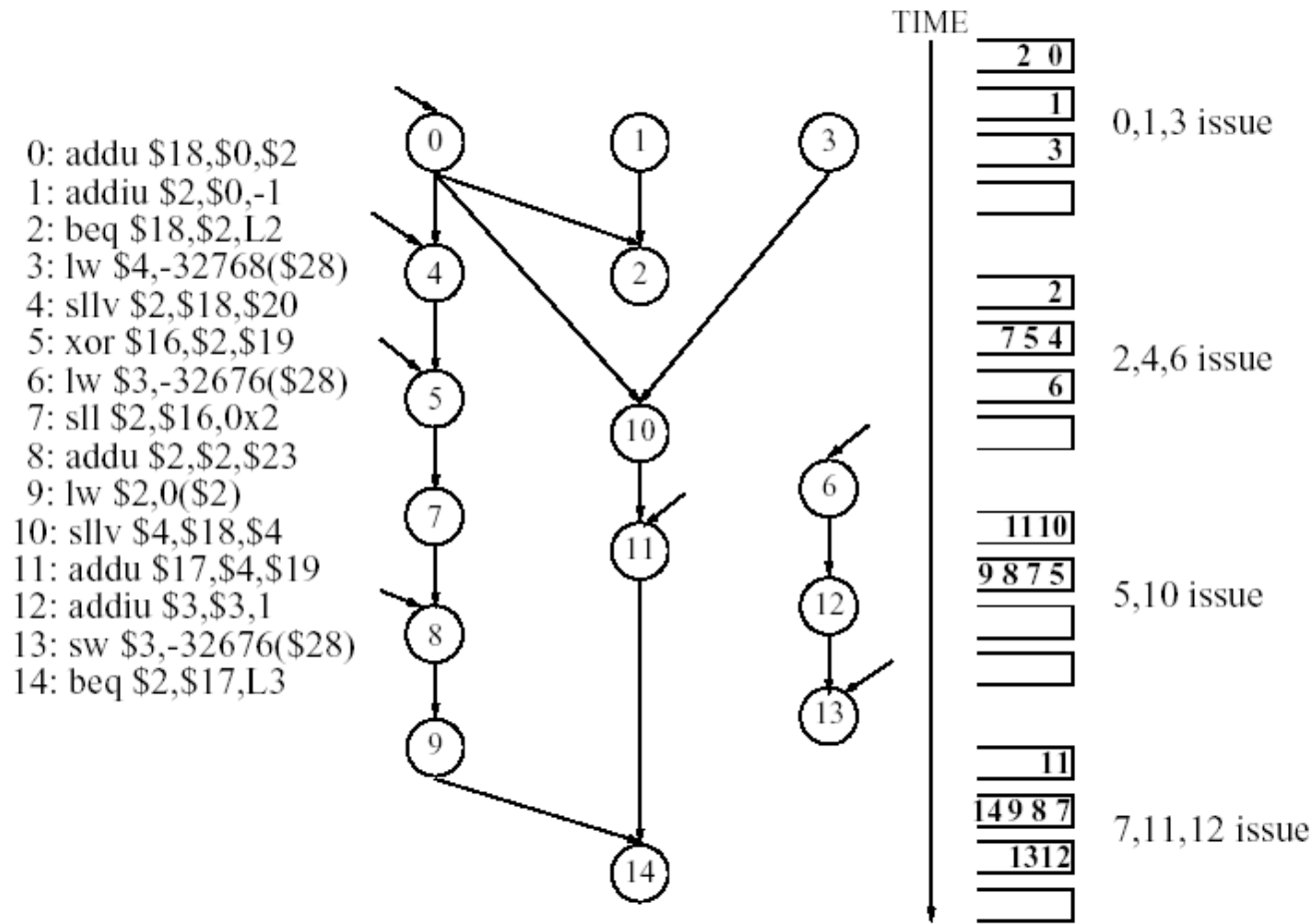
Desktop/Workstation Market



Mobile Market

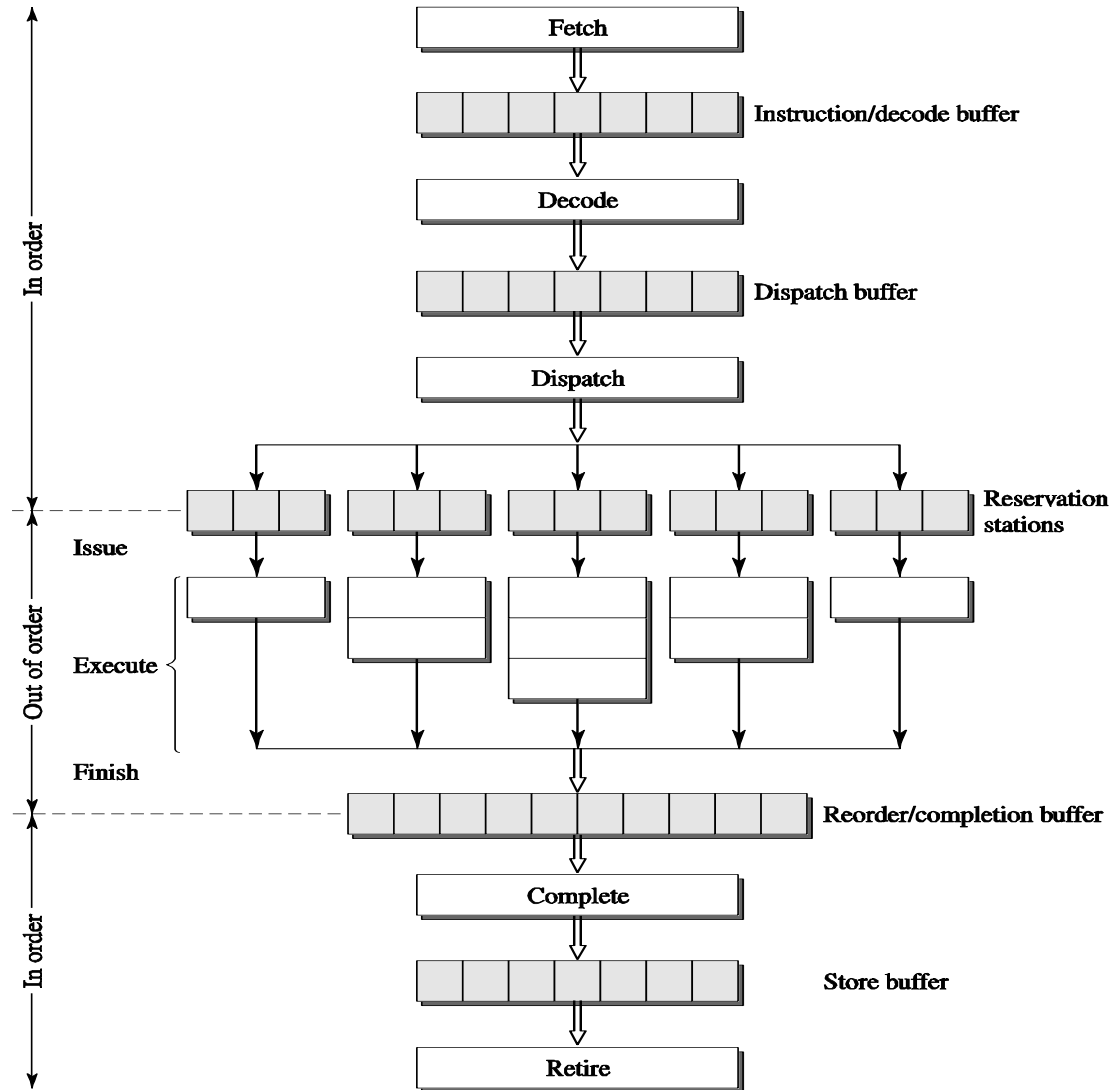


Que fait un processeur à fort IPC ?



1. Fetch & decode
2. Construction graphe de dépendances (DDG)
3. Evaluation DDG
4. Commit (modifier l'état du programme)

Processeur à fort IPC typique



1. Fetch & decode

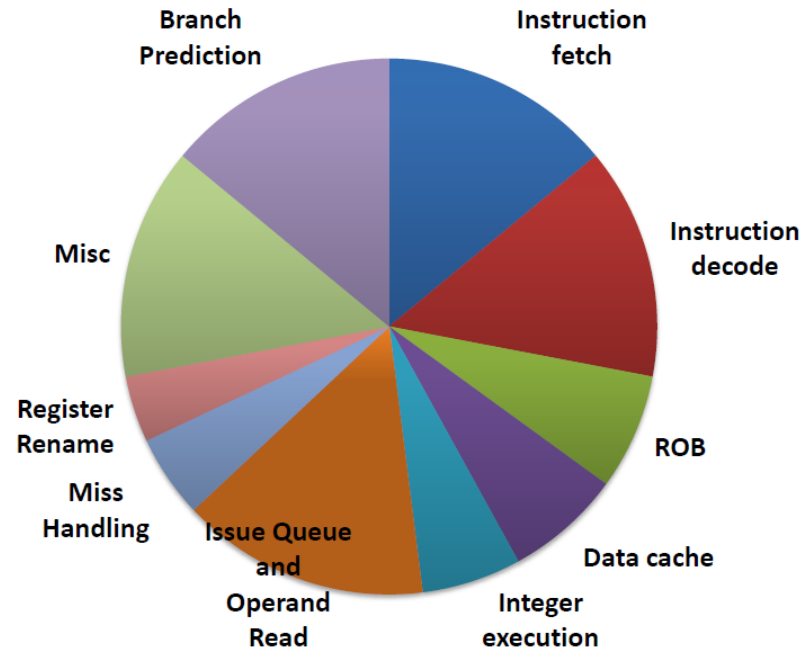
2. Construction graphe de dépendances

3. Evaluation DDG

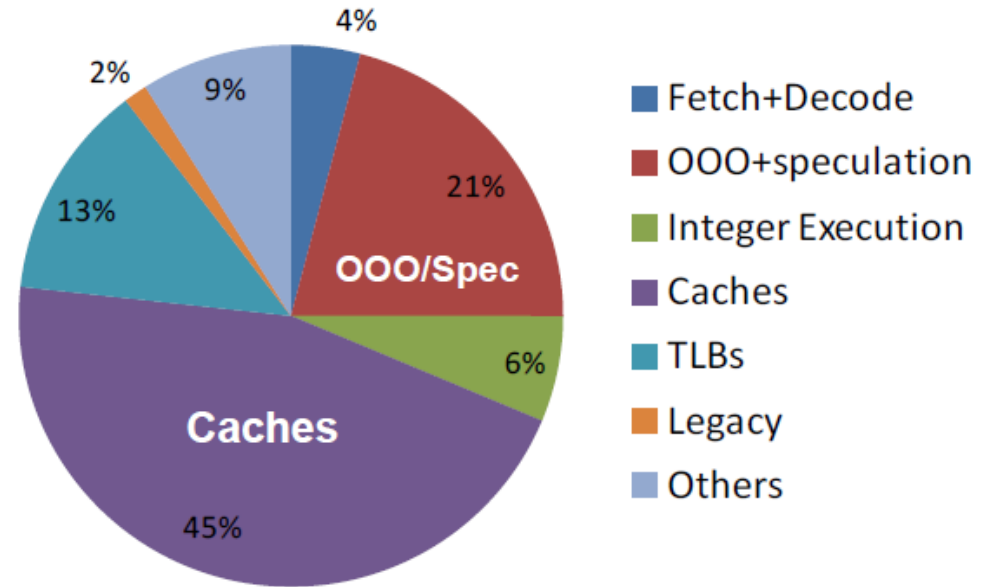
4. Commit

Consommation énergétique

ARM Cortex A15 [Source: NVIDIA]



Core i7 [Source: Intel]



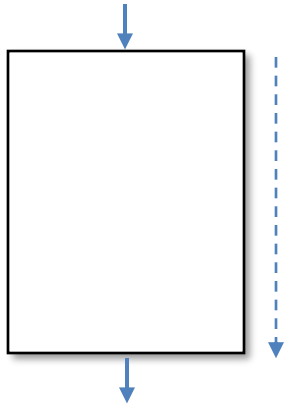
- Le calcul lui-même est submergé par la gestion du pipeline d'exécution agressif.

Limite du pipeline

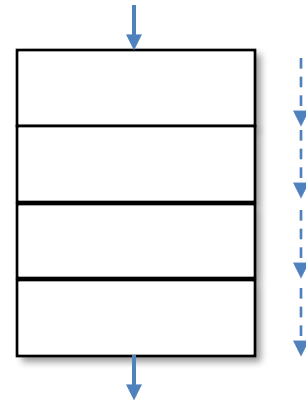
- Bande passante maximum : une instruction par cycle
- Les différents types d'instruction n'ont pas tout à fait le même traitement
 - La latence de la majorité des instructions est plus longue que nécessaire
- Le pipeline est rigide : le gel d'un étage bloque tous les étages antérieurs

Pipelines parallèles

Pas de parallélisme

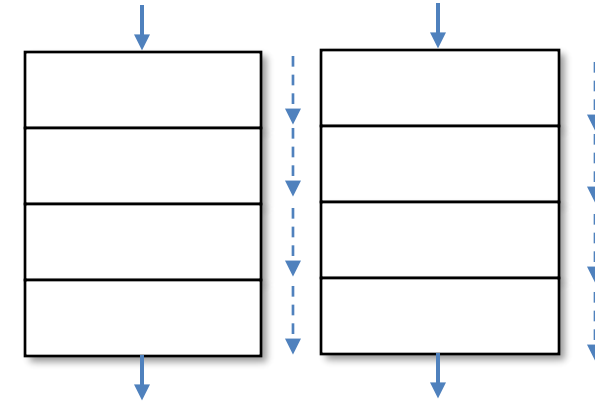


Parallélisme temporel



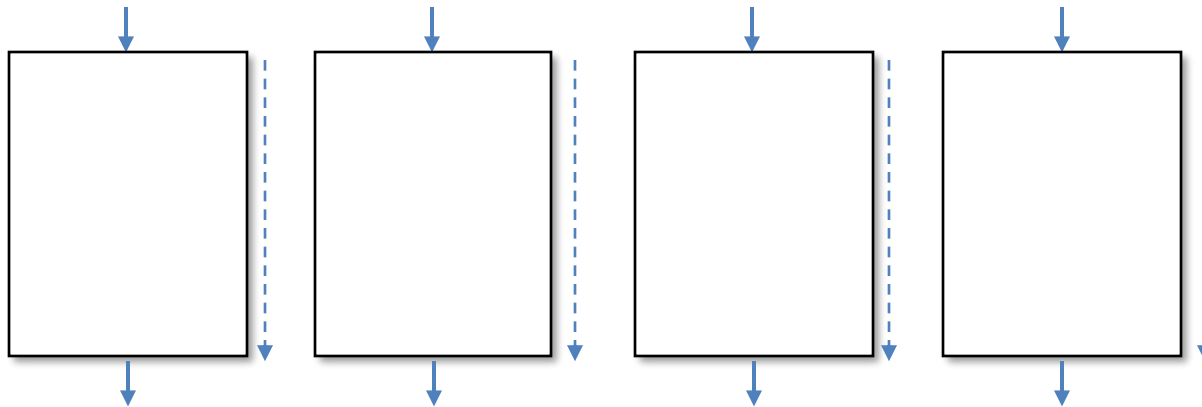
<4

Pipeline parallèle



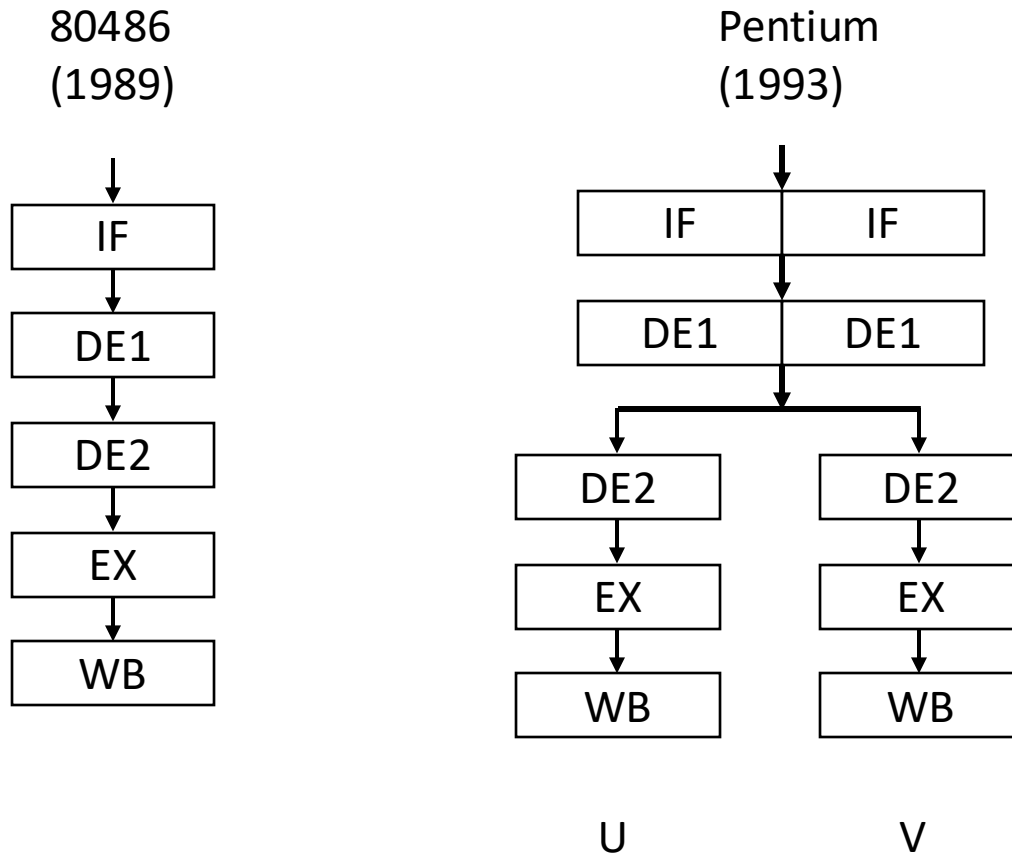
<8

Parallélisme spatial

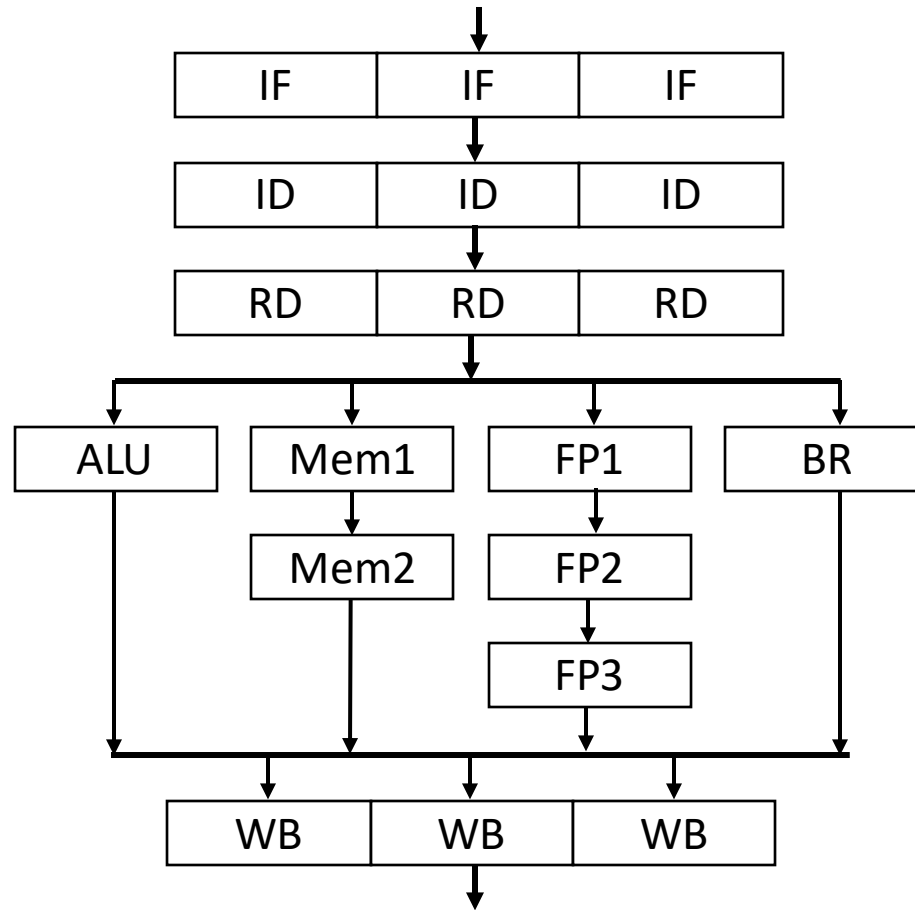


<4

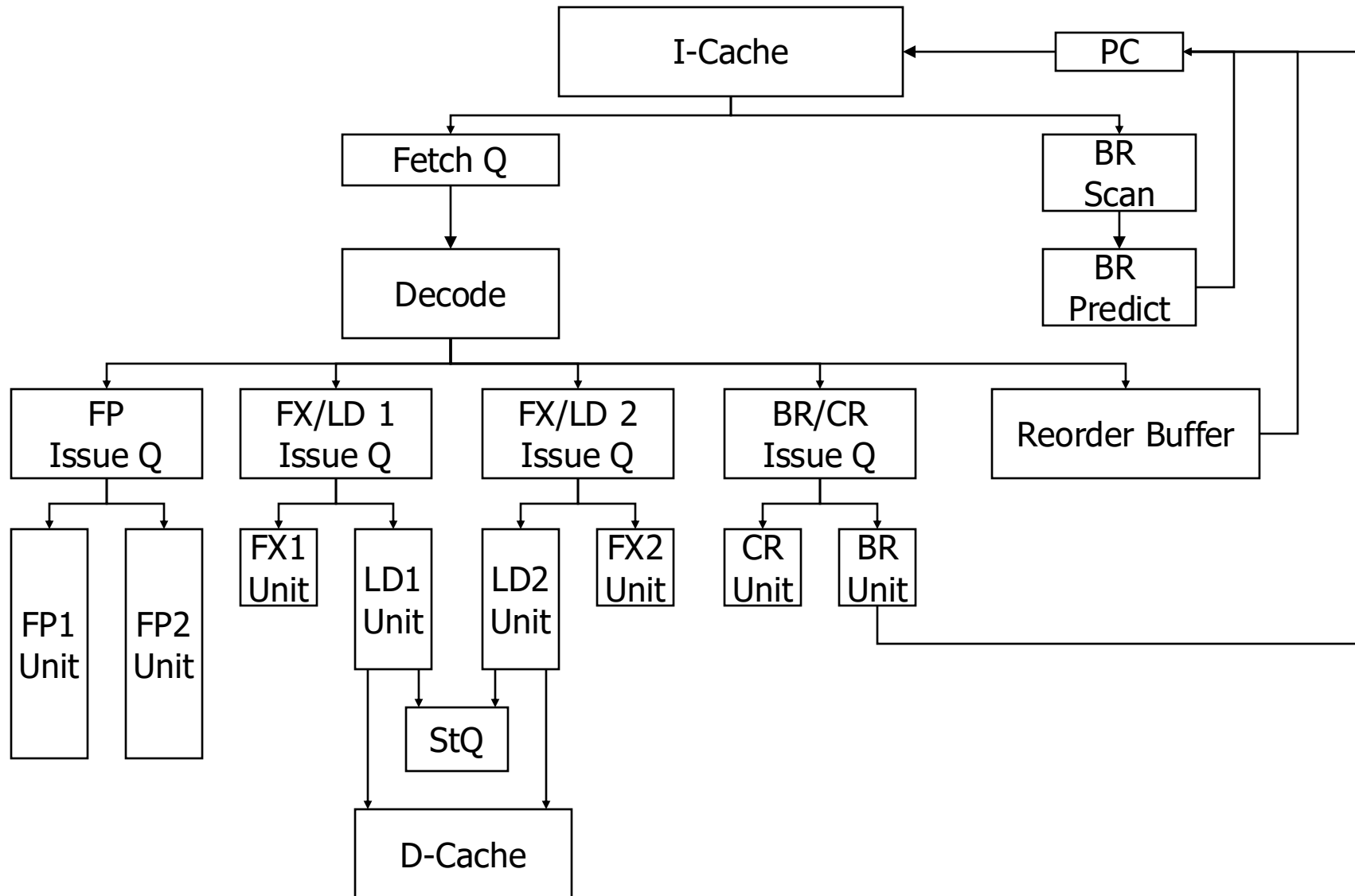
1er pipeline parallèle Intel



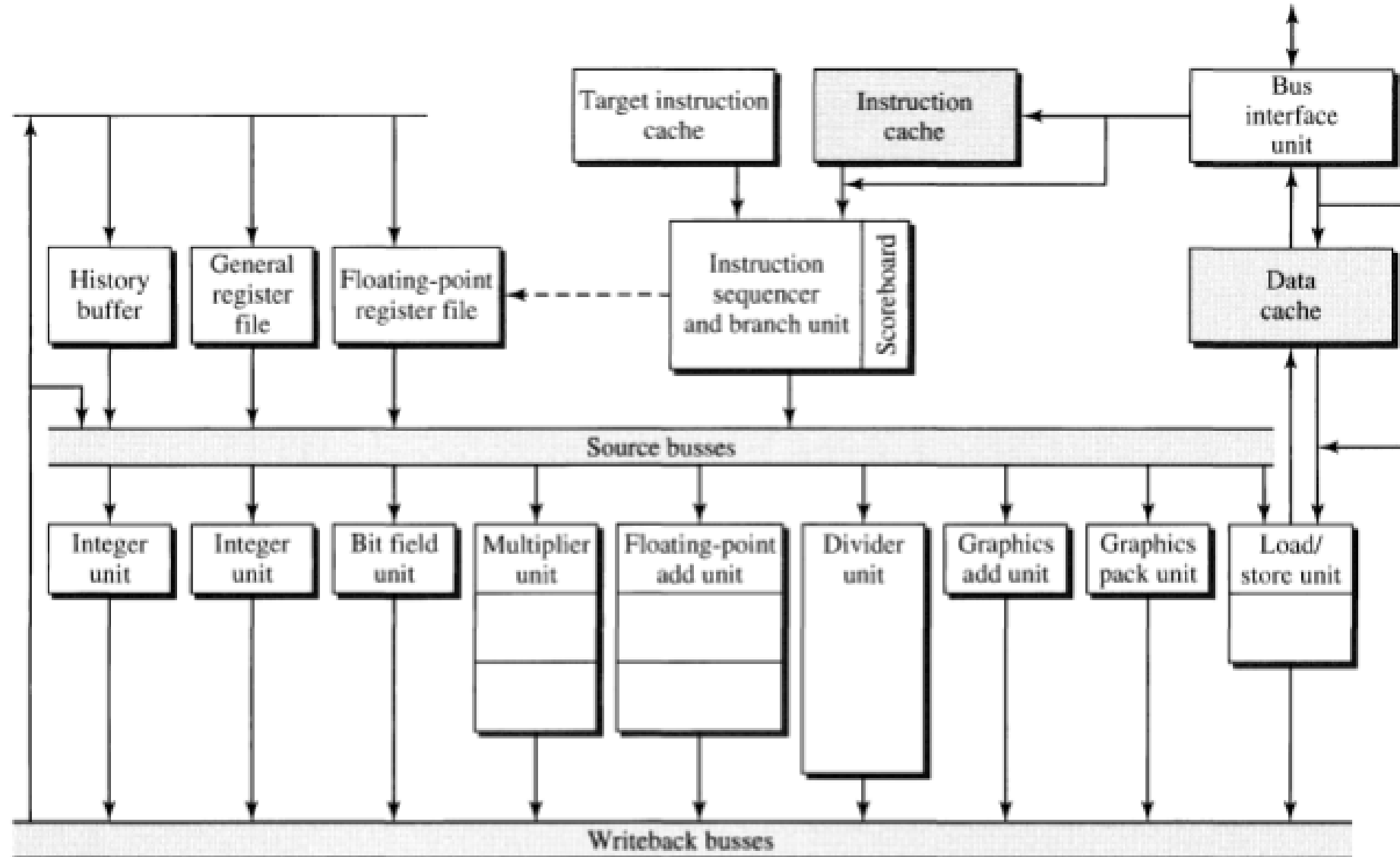
Pipelines diversifiées



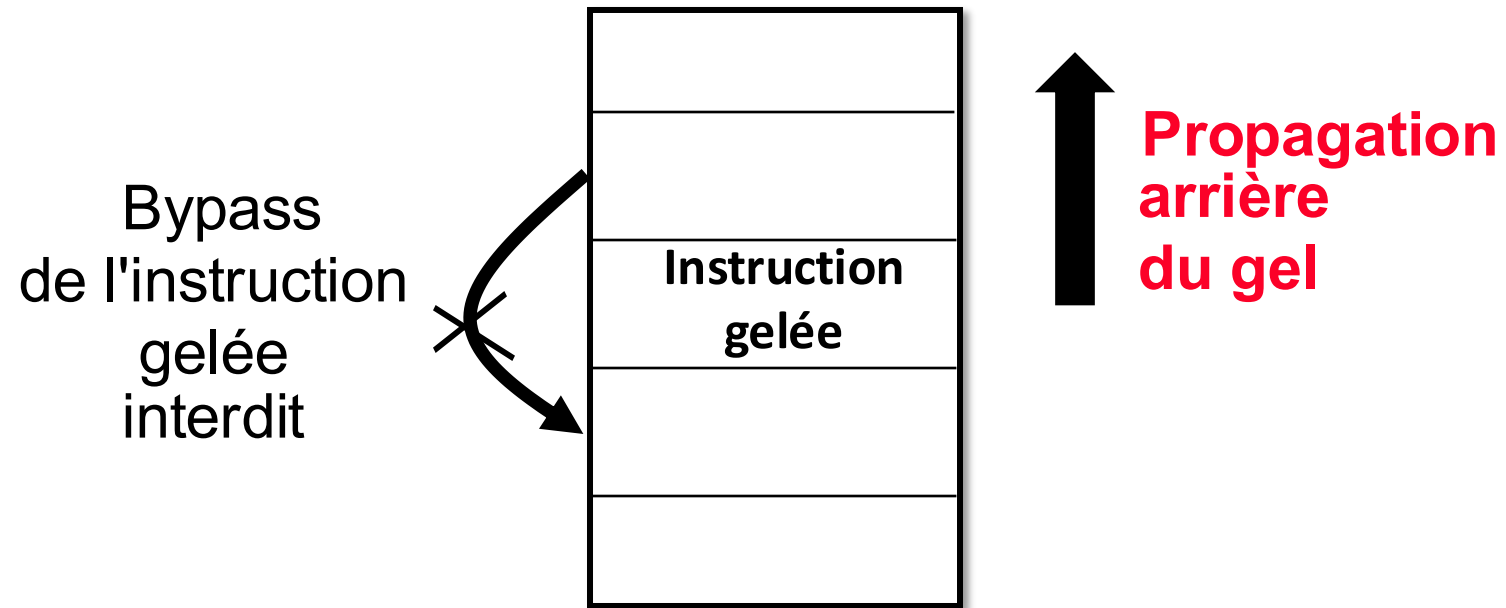
IBM power4



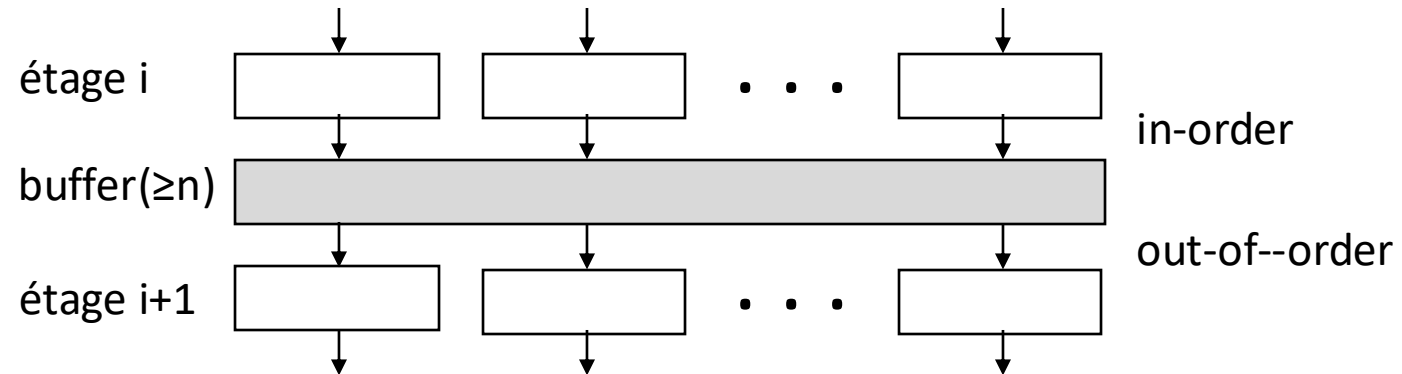
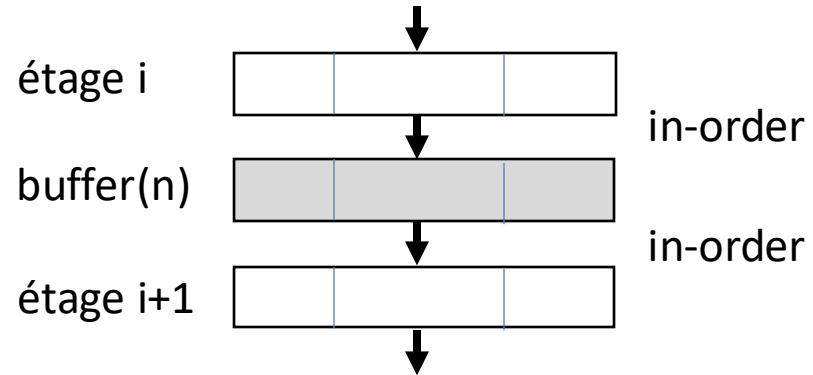
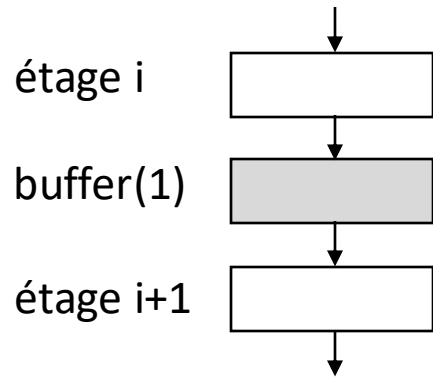
Motorola MC88110



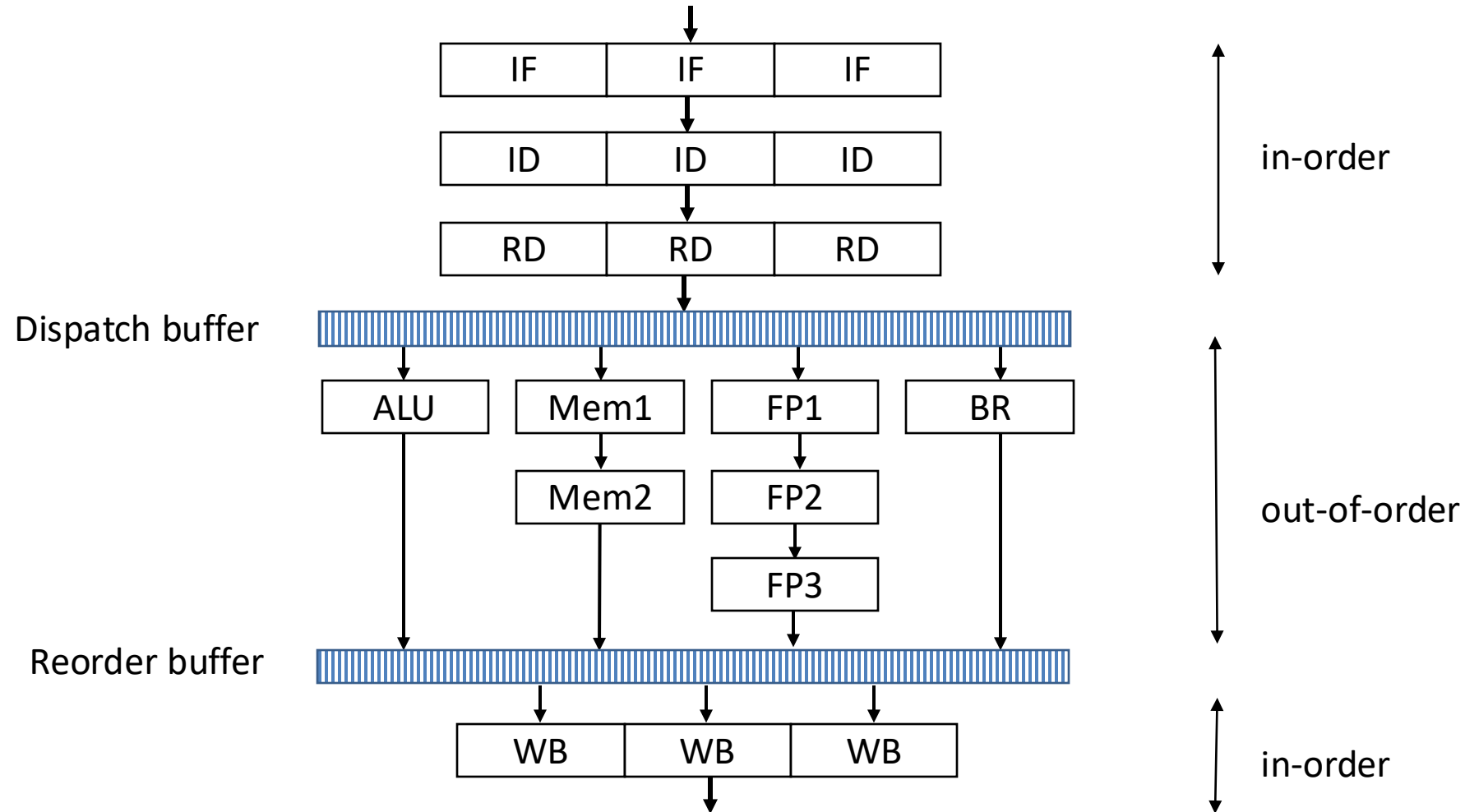
Rigidité du gel du pipeline



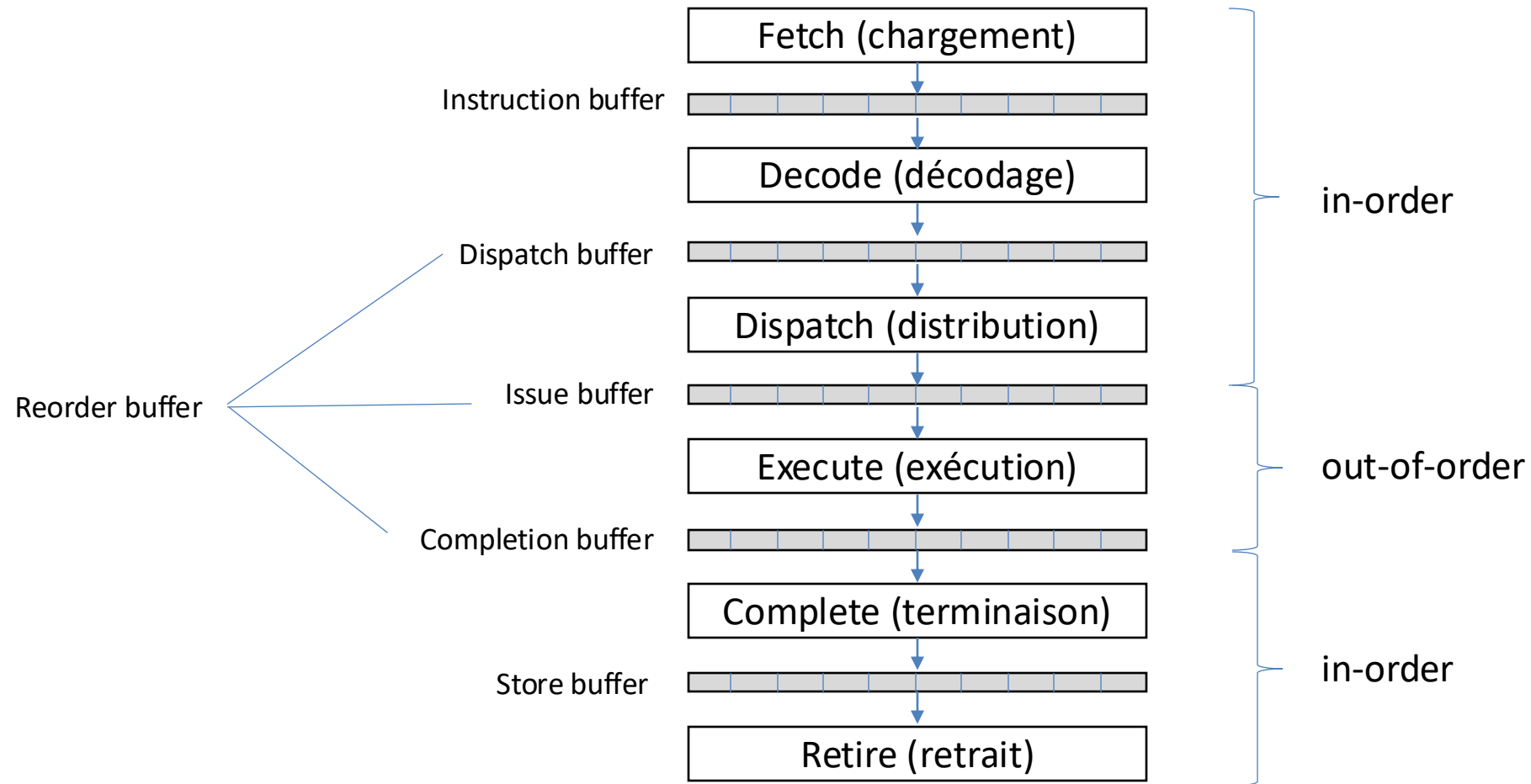
Tampons inter-étages



Pipelines dynamiques

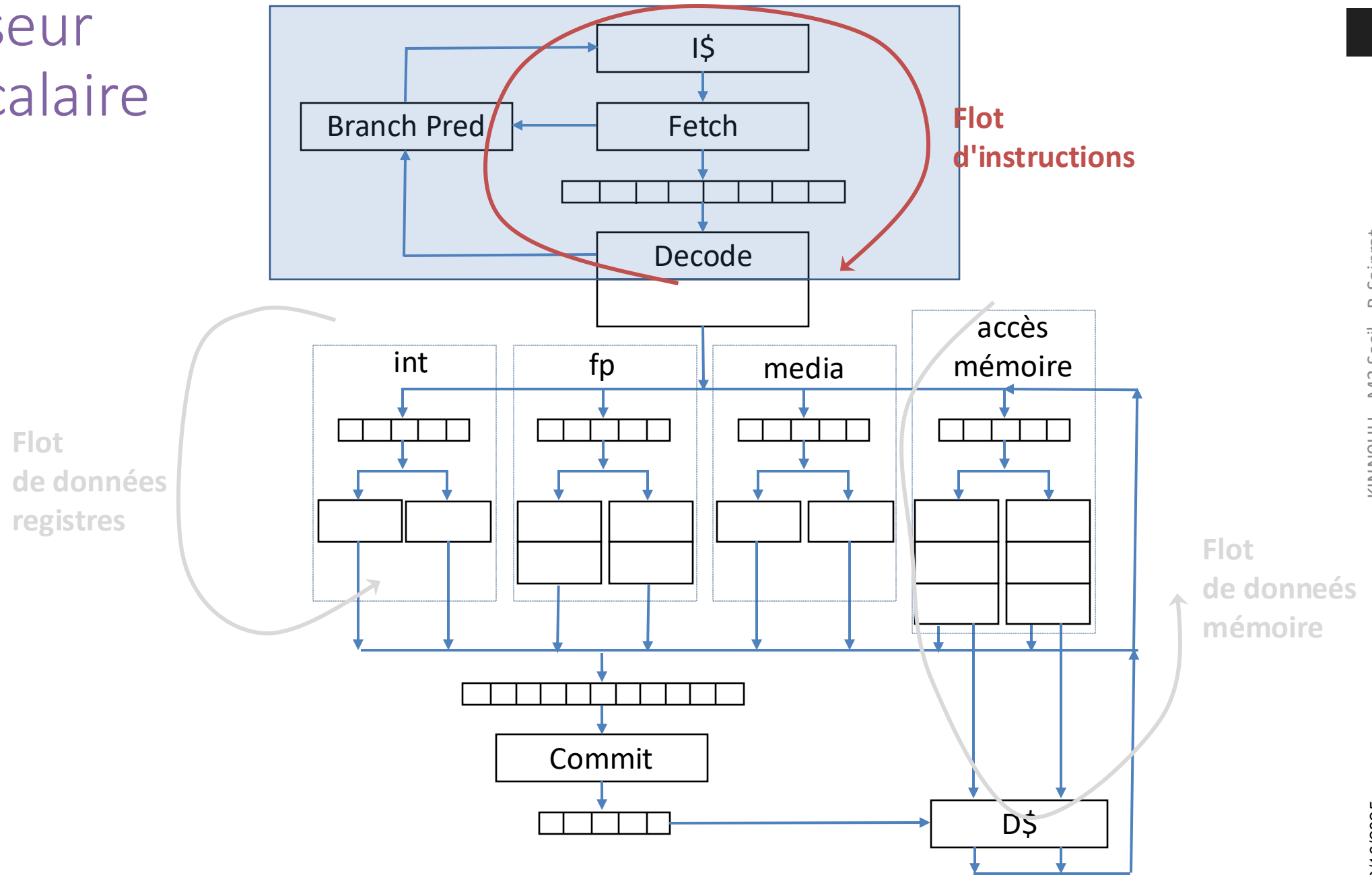


Etapes d'un pipeline superscalaire



Template : les étapes peuvent correspondre à un ou plusieurs étages de pipeline

Processeur superscalaire



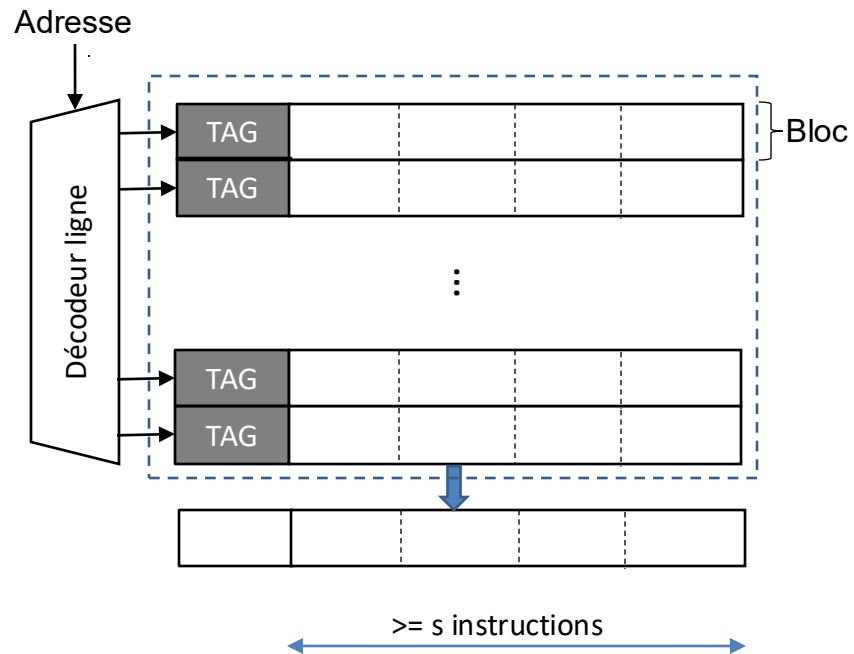
Chargement des instructions (1)

- Le pipeline superscalaire doit être capable de charger plusieurs instructions depuis le cache d'instructions à chaque cycle
- L'étage de chargement des instructions doit maximiser le nombre d'instructions chargées au regard de la largeur du pipeline (s).
 - Idéalement, le processeur doit charger s instructions à chaque cycle.
 - On suppose que le cache de premier niveau a une latence de un cycle et que le processeur charge une ligne physique de la mémoire (qui peut correspondre à un bloc ou une partie d'un bloc)

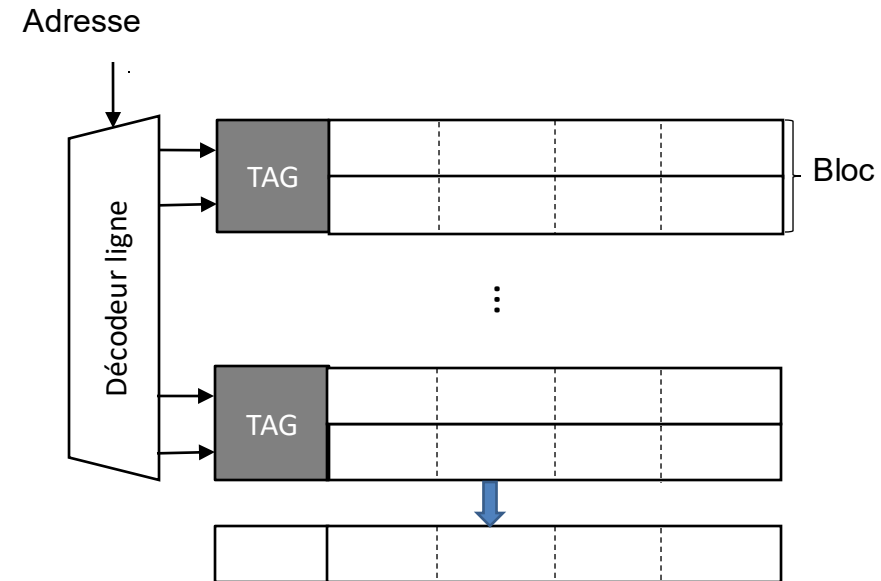
Chargement des instructions (2)

Organisation du cache d'instructions

1 bloc = 1 ligne physique

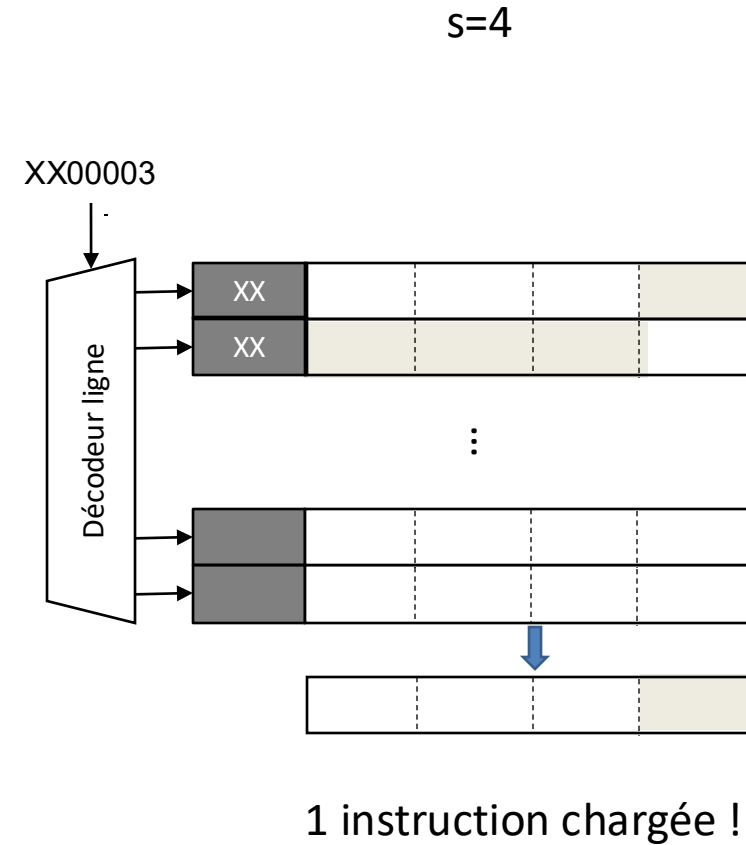
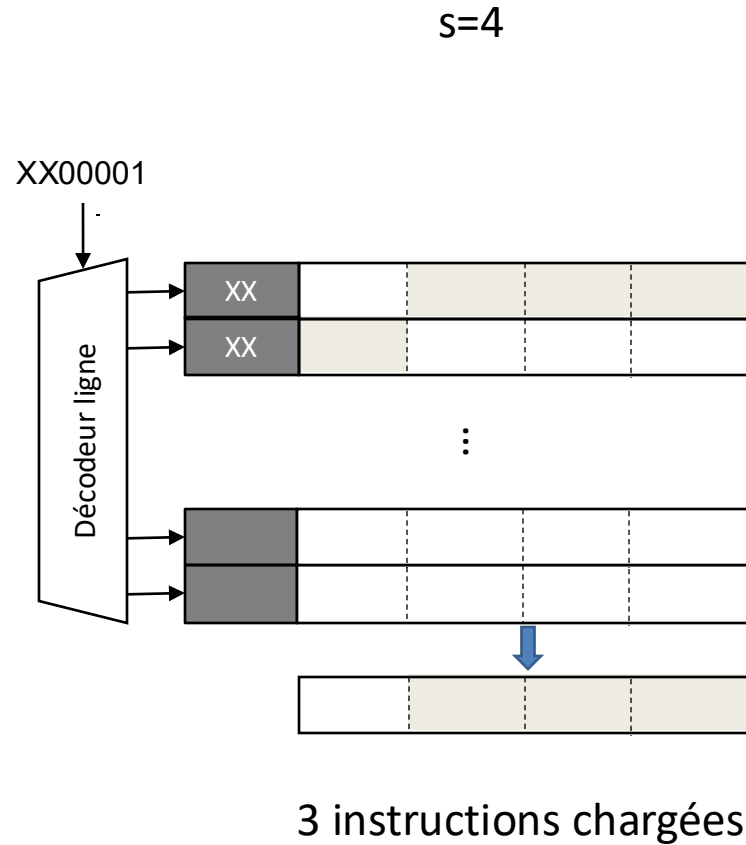


1 bloc = 2 lignes physiques



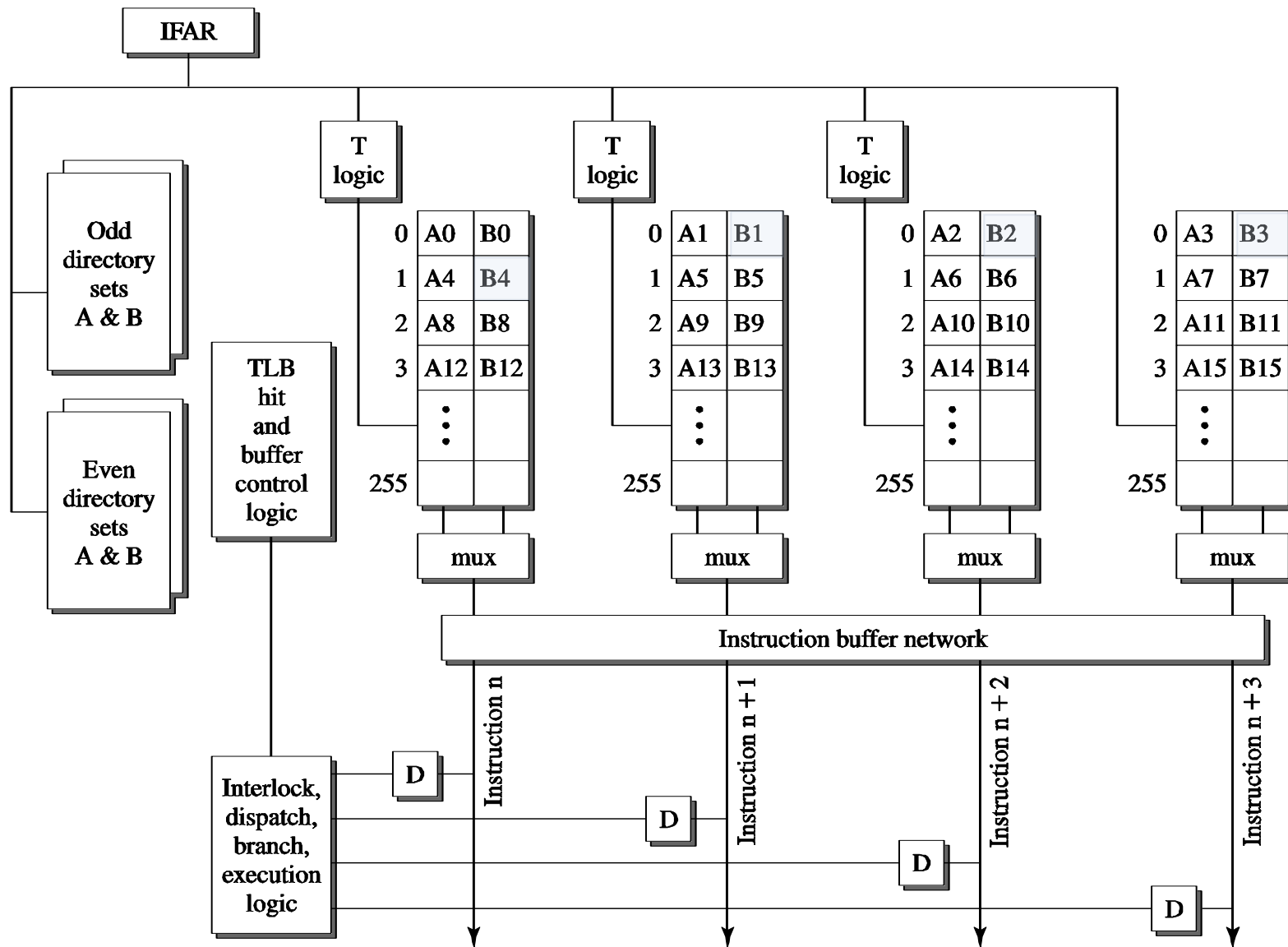
Chargement des instructions (3)

Non alignement du chargement sur les lignes



$$fetch = \frac{1}{4} \times 4 + \frac{1}{4} \times 3 + \frac{1}{4} \times 2 + \frac{1}{4} \times 1 \text{ instructions/cycle}$$

Chargement des instructions (4) Exemple du RS/6000 (IBM, 1989)

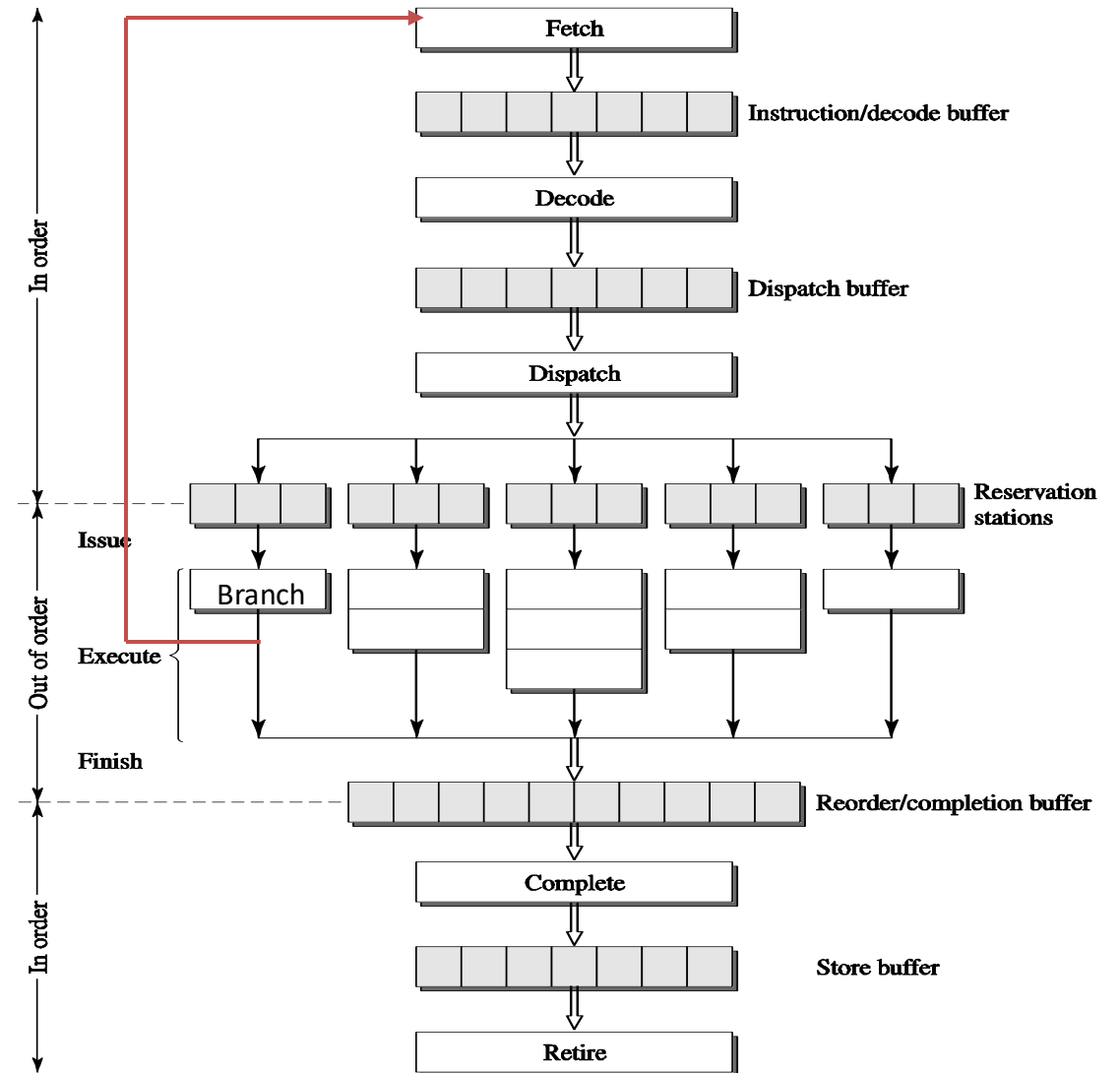


$$fetch = \frac{13}{16} \times 4 + \frac{1}{16} \times 3 + \frac{1}{16} \times 2 + \frac{1}{16} \times 1 = 3,625 \text{ instructions d'une ligne/cycle}$$

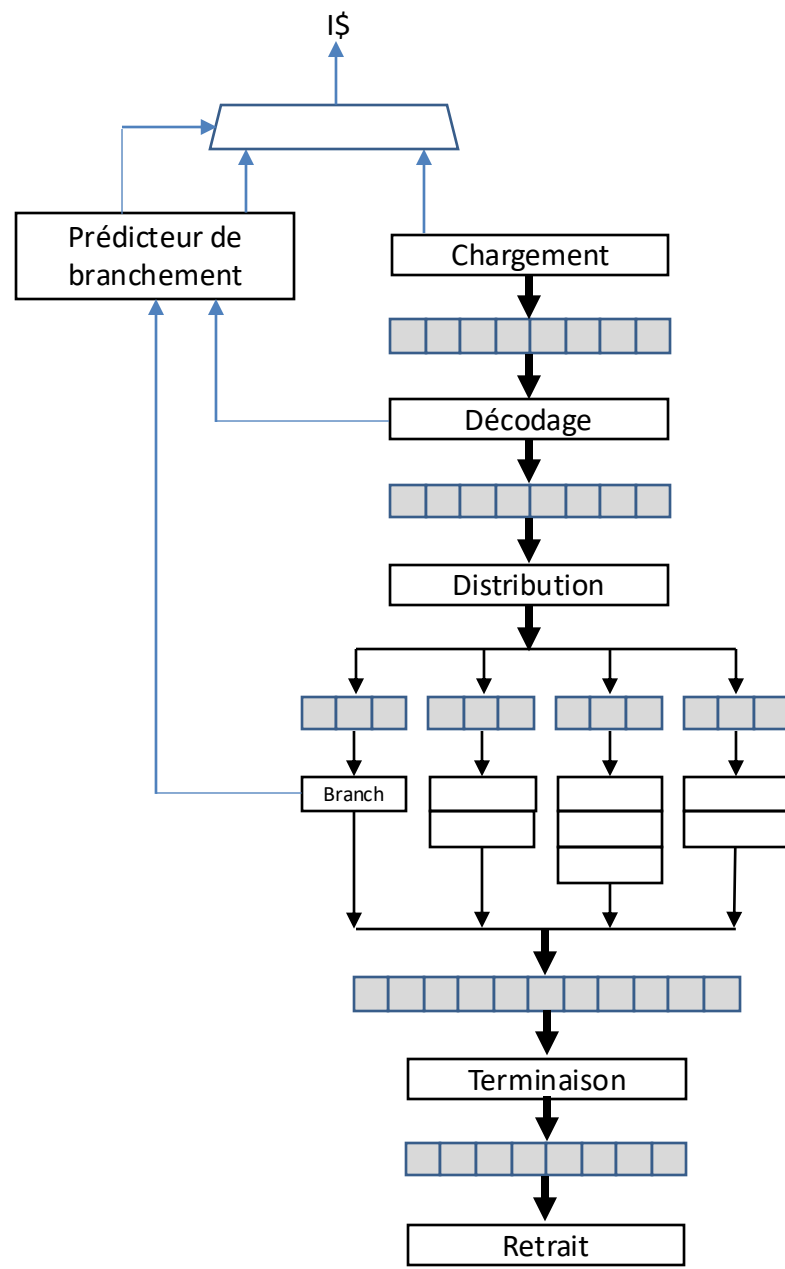
Chargement des instructions (5)

Branchement dans les instructions chargées

- Si un branchement conditionnel ou un branchement inconditionnel est chargé et que la condition est vérifiée, les instructions qui sont chargées en séquence vont être annulées...
- Nécessité de calculer l'adresse cible et de vérifier la condition
 - Beaucoup d'instructions chargées pour rien
 - Prédiction de branchement indispensable
 - Prédiction adresse cible
 - Prédiction condition



Prédiction de branchements



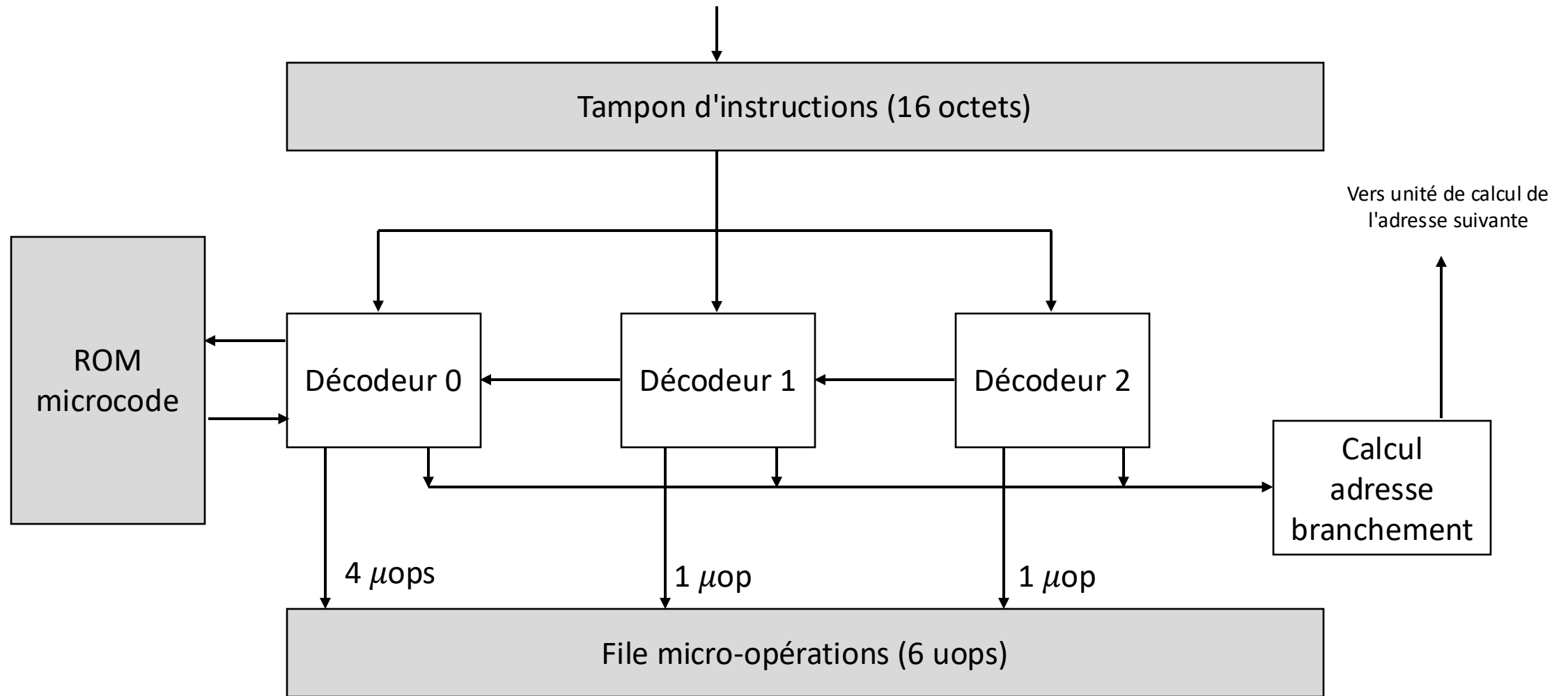
Prédiction de branchements (2)

- Les instructions présentes dans le pipeline peuvent dépendre de plusieurs prédictions successives
 - Importance de bien prédire ($90\% \times 90\% \times 90\% = 73\%$)
- En cas d'erreur
 - éliminer les instructions chargées par erreur
 - Les instructions ont un tag qui indique après quel branchement prédit elles ont été chargées de sorte à libérer les ressources occupées par ces instructions
 - Toutes les instructions dans le decode et le dispatch buffer sont effacées
 - charger les instructions depuis le bon chemin
 - Si le branchement a été prédit non pris, mettre à jour le PC avec la cible calculée
 - Sinon avec l'adresse qui suit celle du branchement
 - Utiliser ce PC pour charger les prochaines instructions puis continuer normalement avec la prédiction de branchement

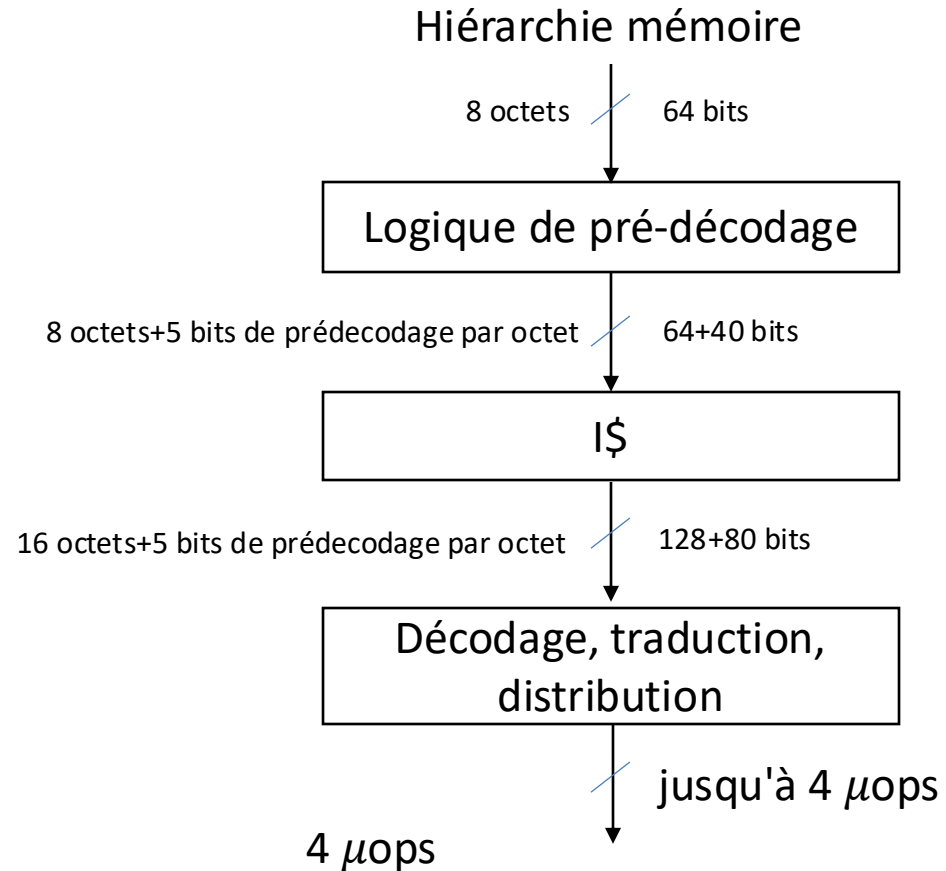
Décodage parallèle

- Tâches
 - Identifier les instructions (facile pour un RISC, pas pour un CISC)
 - Déterminer les types d'instruction
 - Déterminer les dépendances entre instructions
- Deux facteurs importants
 - Jeu d'instructions
 - Largeur du pipeline

Chargement et décodage des Pentium pro et 2



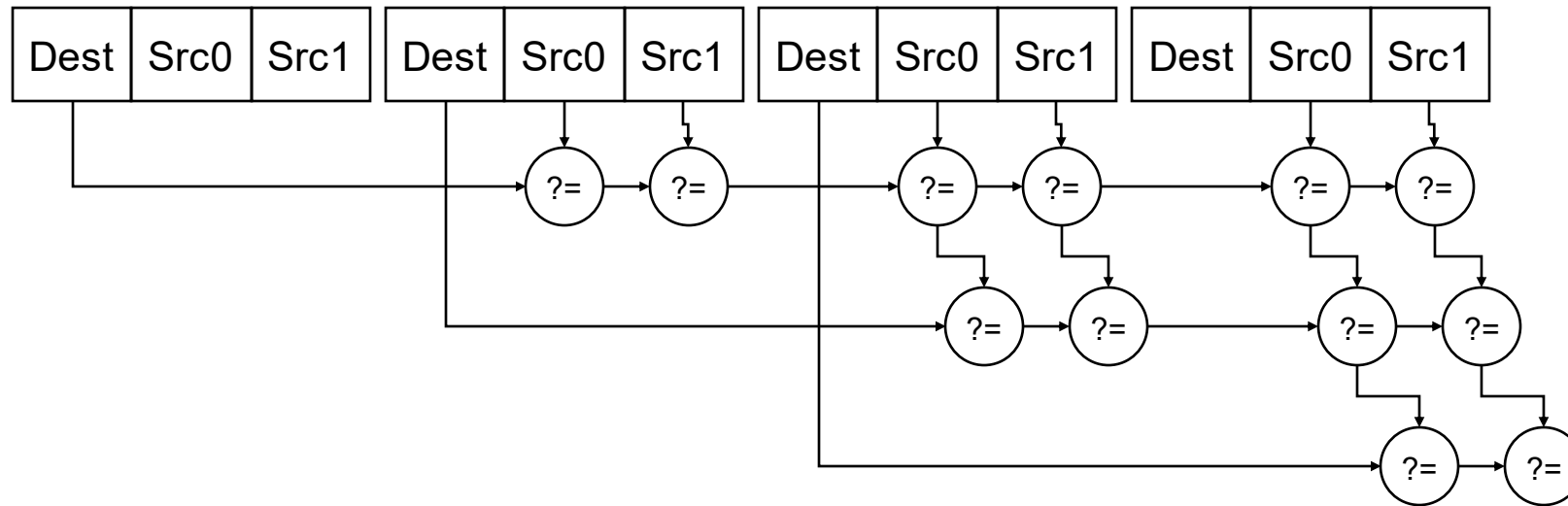
AMD K5 : pré-décodage



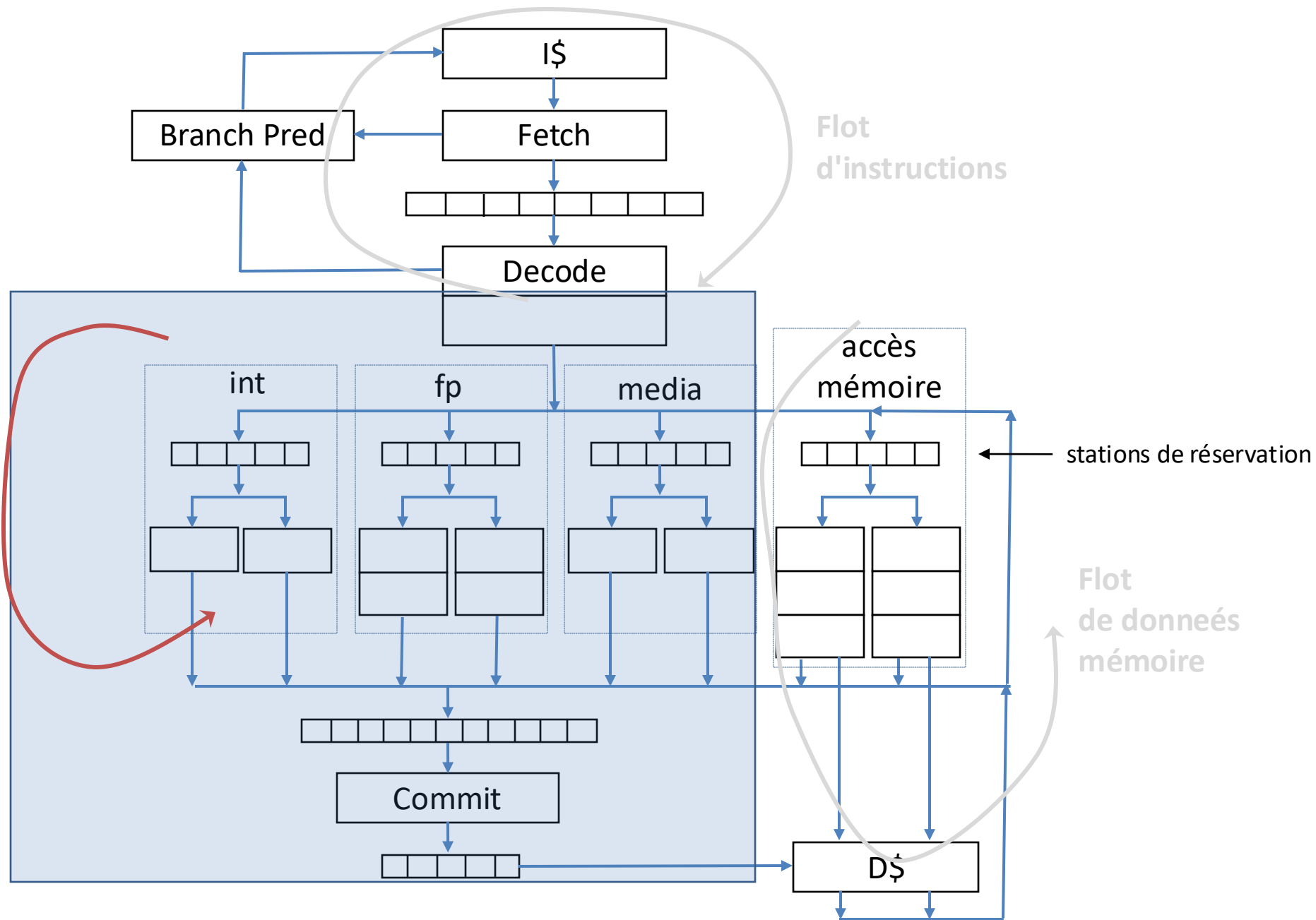
- Utilisé dans certains processeurs RISC pour identifier des branchements plus tôt par exemple ou identifier des groupes d'instructions indépendantes

Contrôle des dépendances

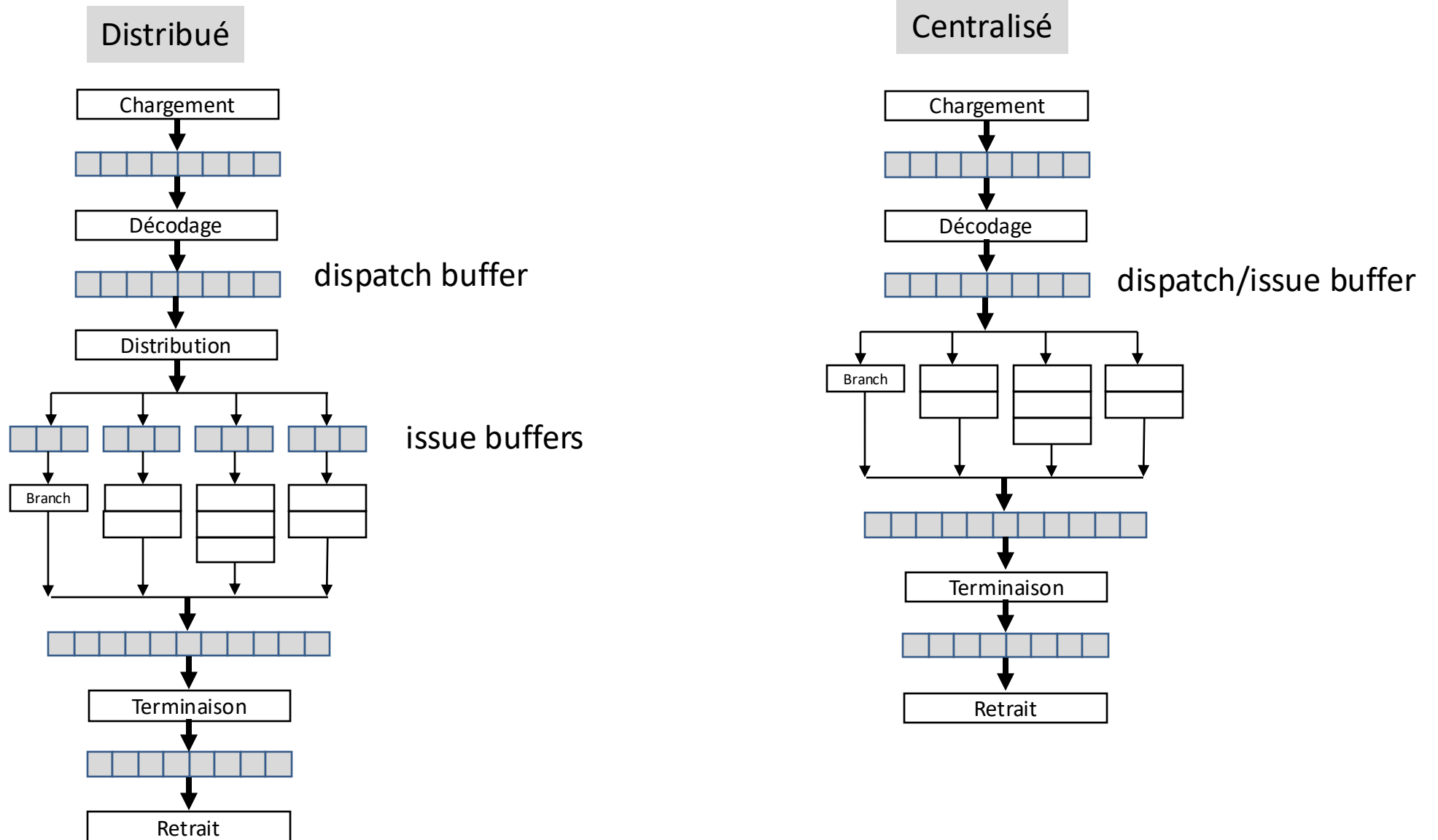
- Pour chaque instruction, sauf la première, rechercher si chacun des opérandes est produit par une instruction précédente.



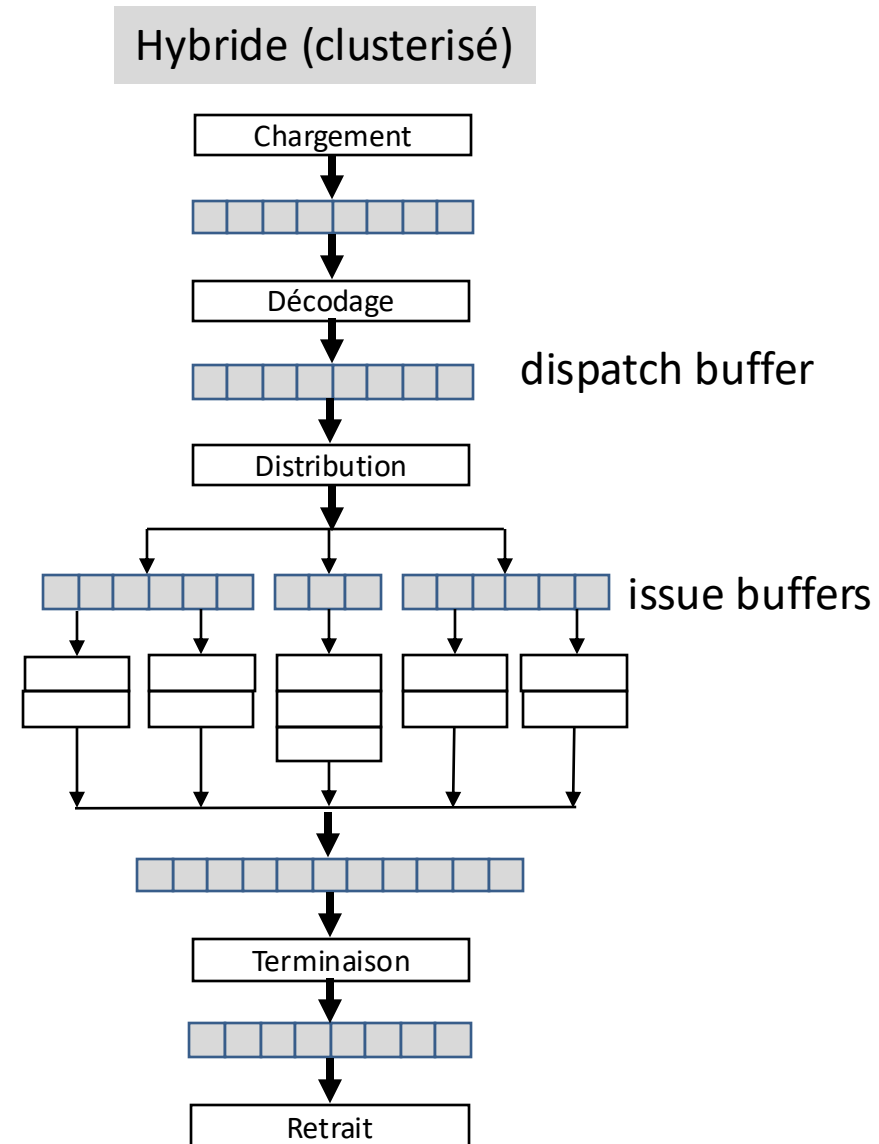
Flot
de données
registres



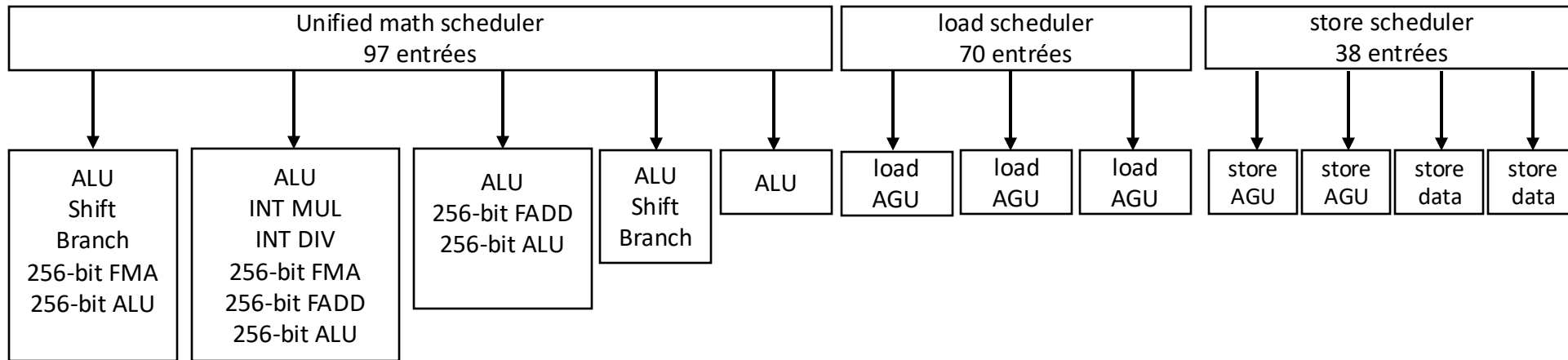
Reservation stations (issue buffer)



Reservation stations (2)



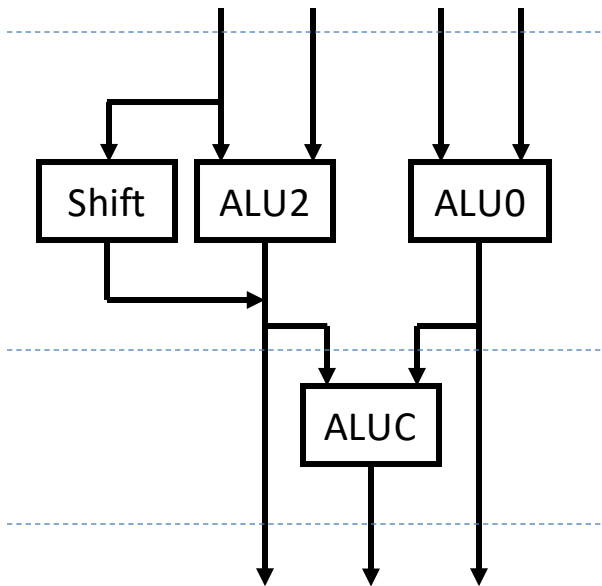
Golden Cove (11/2021)



Exécution

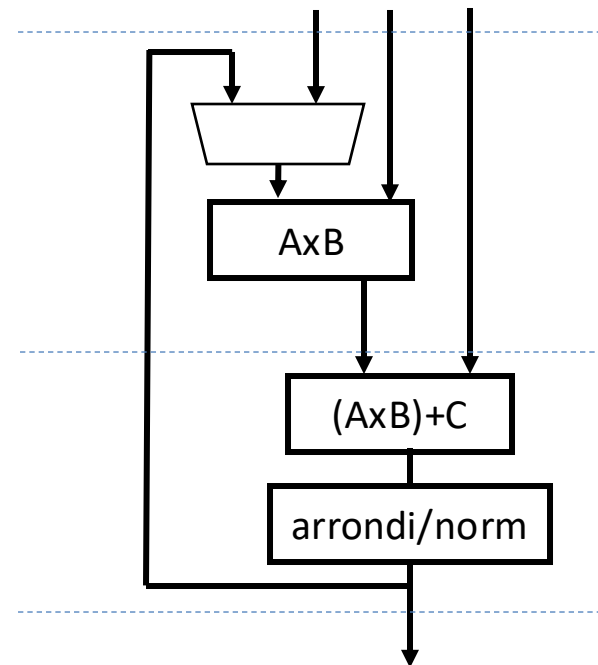
- Le cœur du processeur
- Plus d'UF et plus spécialisées

TI supersparc



2 opérations indépendantes ou non
en // mais en 2 cycles

IBM RS/6000



Fused-Multiply-add
L'addition est $Ax1+C$,
la multiplication $AxB+0$

Intel golden cove

ALU 256 bits :
instruction set vector extensions (AVX)
Registres spécifiques
8 additions 32 bits en parallèle ou
32 additions 8 bits en parallèle ou
...
= SIMD
Difficulté à programmer
=> Bibliothèques
Traitement multimedia,
traitement de signal,
calcul intensif
IA

Quel mix d'UF ?

- Programs :
 - 40% ALU,
 - 20% branchements,
 - 40% load/store
- Processeurs :
 - 2 voire 3 ALU
 - 1 seul port sur le cache

Processor	PPC 604	MC 88110	Ultra Sparc	DEC 21164	MIPS 10000	
Date (ship)	94	92	95	95	95	
Degree	4	2	4	4	4	
Integer unit	2	2	2	2	2 ³	
Shift unit		1				
Divide unit	1	1 ¹				
Multiply unit		1 ²				
FP add unit	1	1	2	1	1	
Convert unit		1 ¹			1	
FP divide unit		1 ²				
FP multiply unit						
Data cache port	1	1	1	2	1	

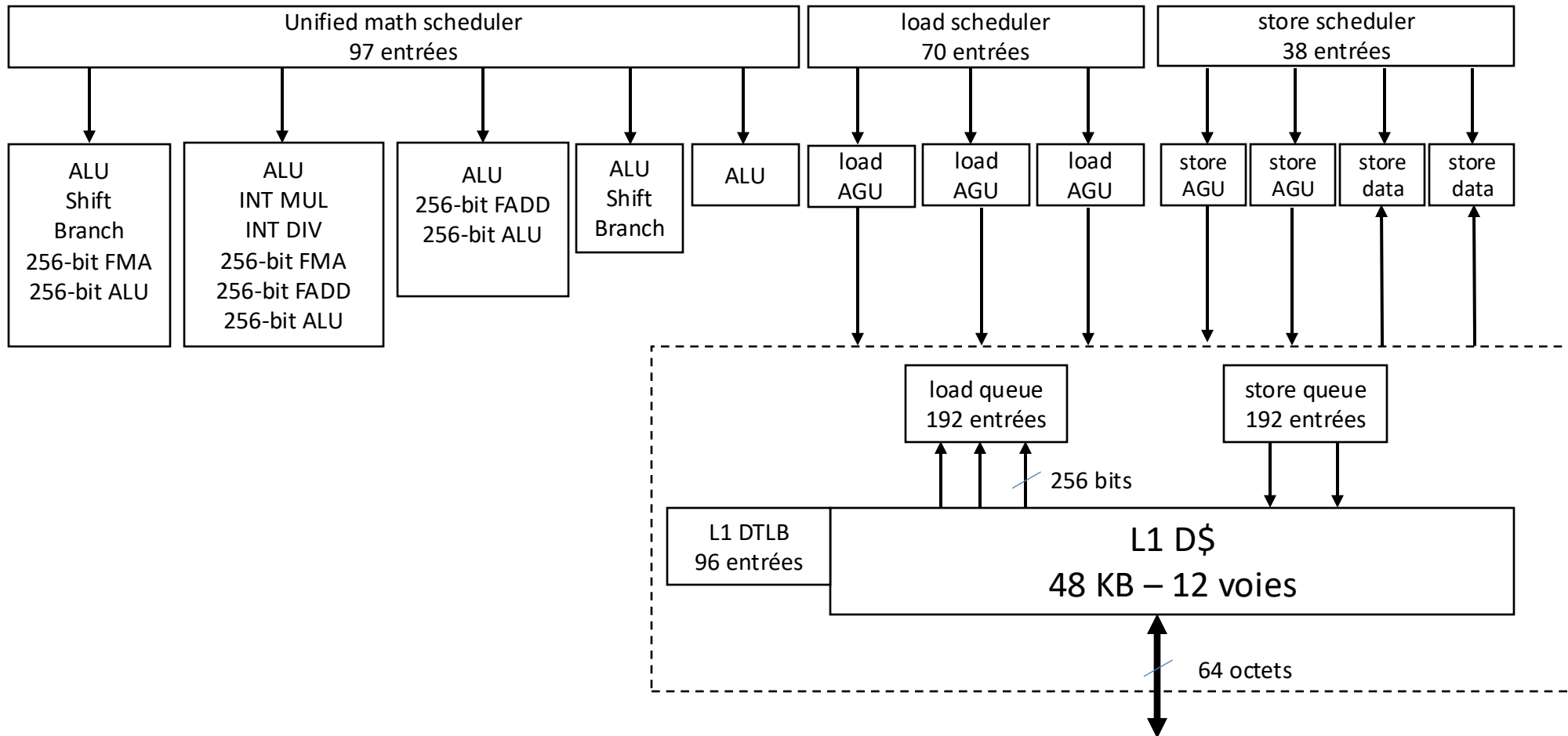
Table 1 — *Configuration of Functional Units*

¹ Dividers are merged in the same functional unit

² Multipliers are merged in the same functional unit

³ Both units handle integer instructions but only one processes shifts while the other processes divides and multiplies.

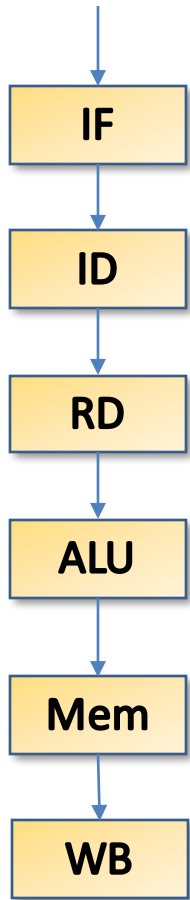
Quel mix d'UF ?



Les dépendances

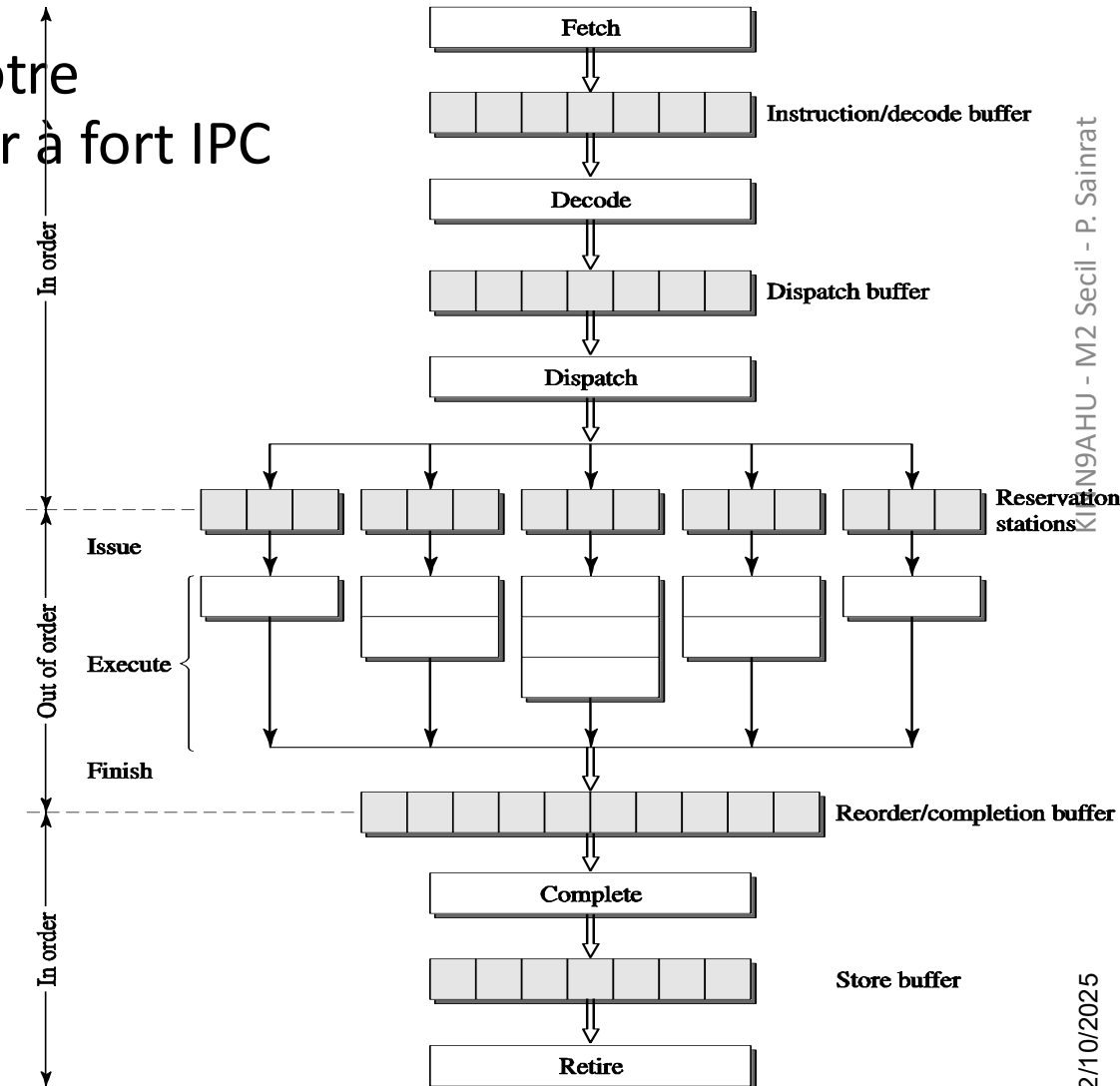
- Vraies
 - j ne peut pas s'exécuter tant que i n'a pas produit son résultat
- Anti
 - j ne peut pas écrire son résultat tant que i n'a pas lu ses sources
- de sortie
 - j ne peut pas écrire son résultat tant que i n'a pas écrit son résultat

Analyse des aléas dans un pipeline scalaire



- Pipeline scalaire
 - Aléas mémoire
 - EAL: Oui/Non?
 - EAE: Oui/Non?
 - LAE: Oui/Non?
 - Aléas registres
 - EAL: Oui/Non?
 - EAE: Oui/Non?
 - LAE: Oui/Non?

- Et dans notre processeur à fort IPC typique ?



Dépendances de données sur les registres

- Dans un processeur superscalaire OOO, les trois types de dépendances peuvent causer des aléas puisque les lectures et les écritures sont dans le désordre.
- Une seule solution pour les trois : renommage de registre
 - Pour chaque écriture dans un registre architectural, attribution d'un registre physique.
 - Les écritures se font dans l'ordre à l'étage de write-back (assuré par le reorder buffer)
 - WAR et WAW ne sont plus un problème, les lectures se font forcément avant le write-back.

R3 <= ...	P32 <= ...
R4 <= ...	P33 <= ...
R3 <= ...	P35 <= ...

Renommage de registres : LAE

- Au dispatch
 - Table de renommage indique si un opérande est en cours de production et fournit le numéro du registre physique
 - Sinon, lecture dans le banc de registre
 - Allocation d'un nouveau registre pour le registre destination

$R3 \leq R2 + R1$ $P32 \leq P2 + P1$

$R4 \leq R3 - R1$ $P33 \leq P32 + P1$

$R3 \leq R4 \& R2$ $P35 \leq P33 + P2$

$R5 \leq R2 - R1$ $P36 \leq P2 - P1$

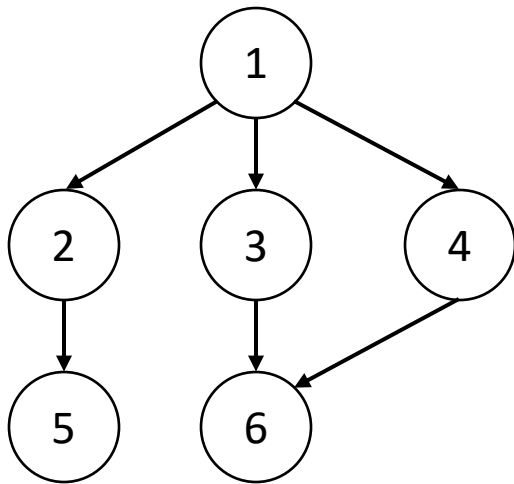
- Lancement (issue) : les instructions attendent leurs opérandes dans la file de lancement (stations de réservations ou issue buffer)
 - Des instructions plus récentes peuvent dépasser de plus anciennes

Ordonnancement des instructions

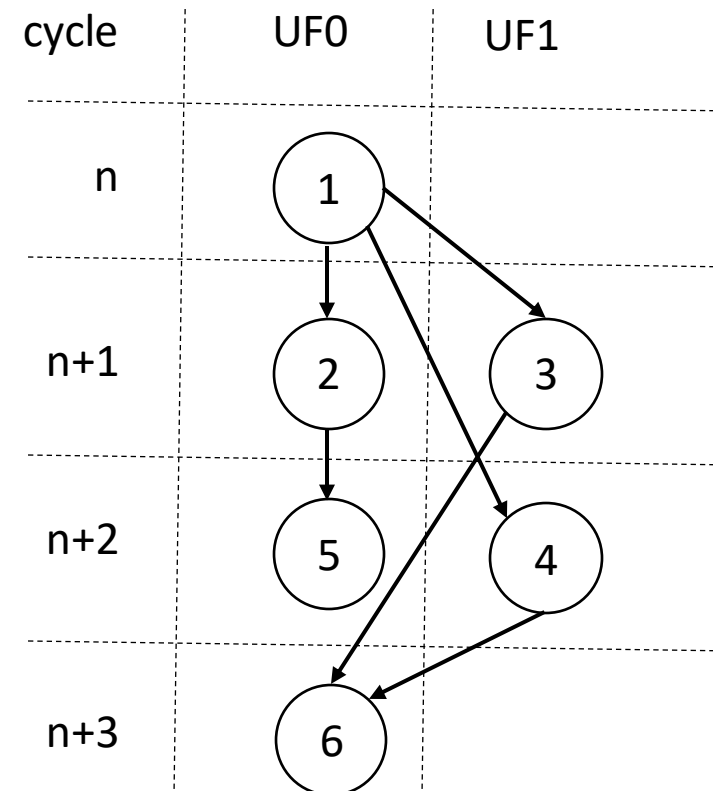
instruction scheduling

- Faire correspondre une série d'instructions aux ressources d'exécution (UF)
 - Quand et où une instruction doit-elle être exécutée ?

Graphe de dépendances



Exécution sur deux UF

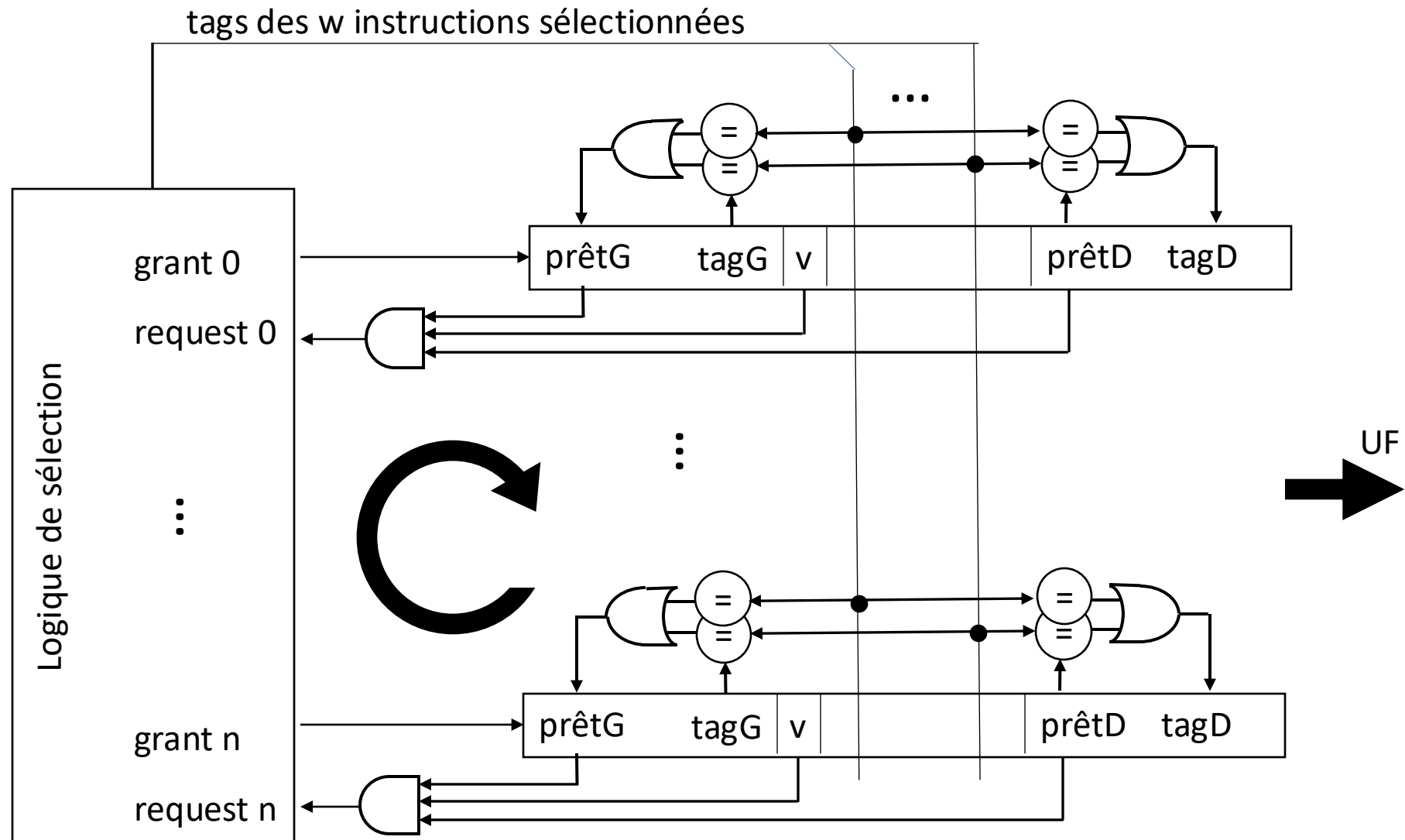


Ordonnancement des instructions

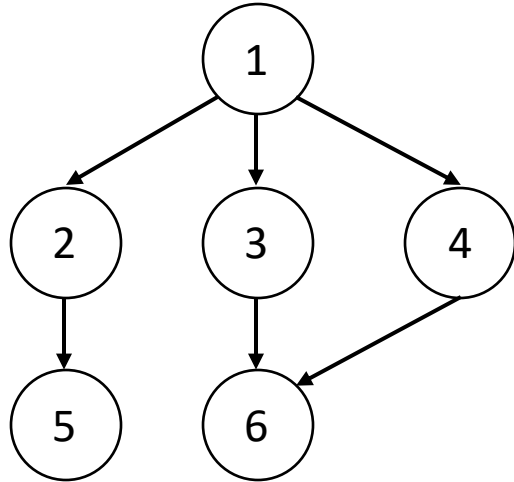
- Un ensemble d'opérations de réveil et de sélection
- Réveil
 - Diffusion du tag des instructions productrices sélectionnées
 - Instructions en attente de ce tag déterminent si leurs opérandes sont prêts
 - Résout les vraies dépendances de données
- Sélection
 - Sélection des instructions à lancer parmi le pool des instructions prêtes
 - Résolution des conflits de ressources
 - Bande passante de lancement
 - Nombre limité d'UF et de ports mémoire

Boucle d'ordonnancement

- Exemple simple de sélection et de réveil

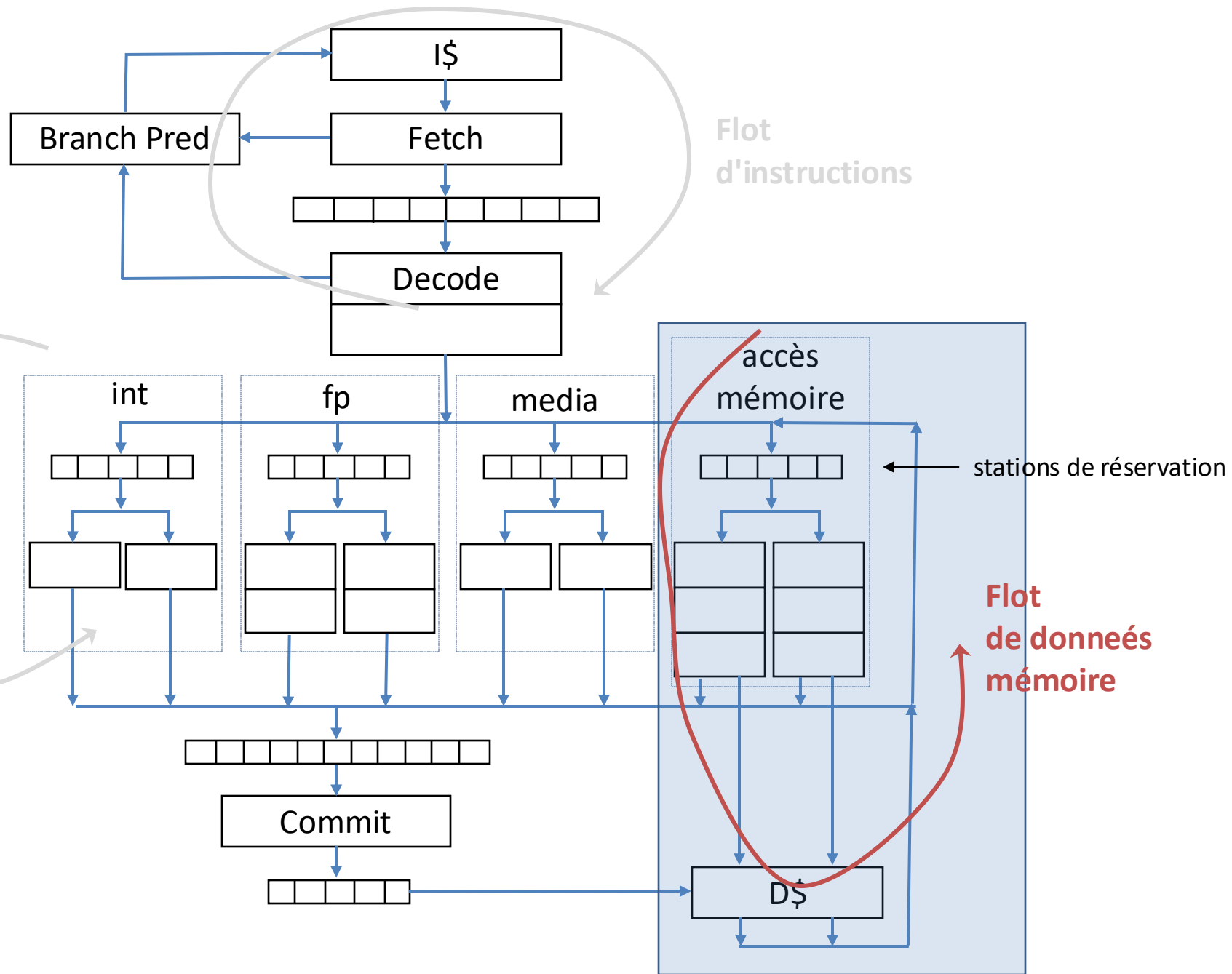


réveil/sélection



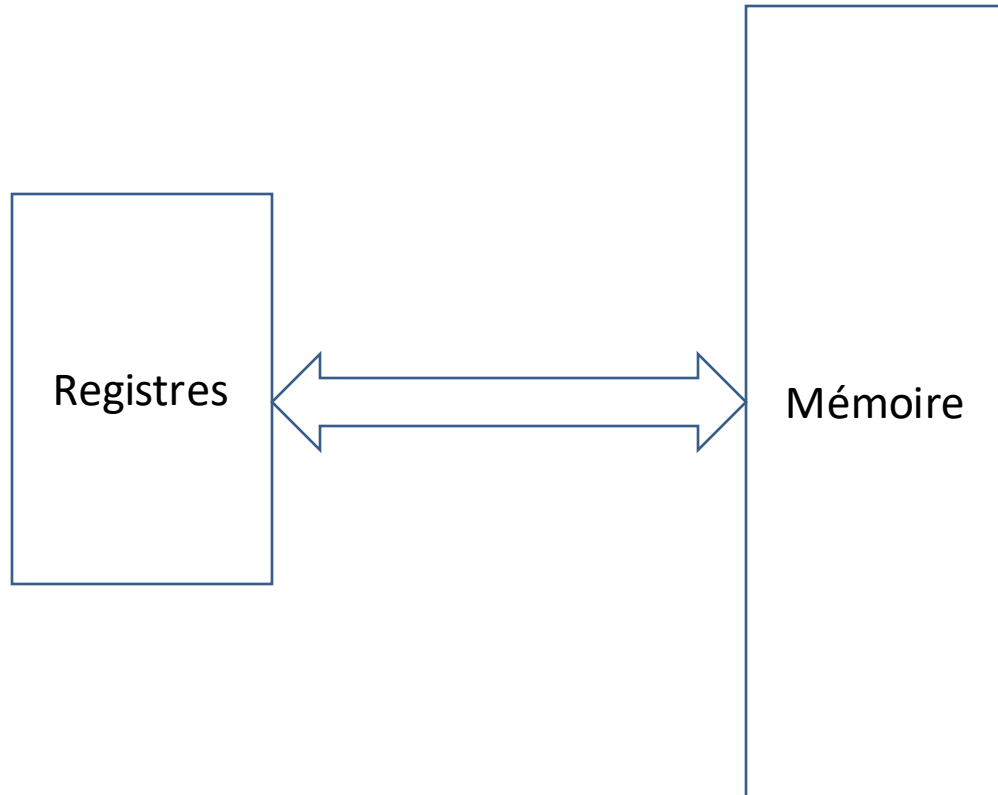
cycle	UF0	UF1	inst. prêtes	réveil/sélection
n			1	s 1 r 2,3,4
n+1			2,3,4	s 2,3 r 5,6
n+2			4,5	s 4,5 r 6
n+3			6	s 6

Flot
de données
registres



Flot de données mémoire

- Résoudre les dépendances mémoire (EAL/EAE/LAE)
 - Si les opérations sont dans le désordre, risques d'aléas
- Fournir une grosse bande passante sur la hiérarchie mémoire
 - Caches non bloquants



3 opérations

Calcul adresse (registre + offset)

Traduction adresse (MV)

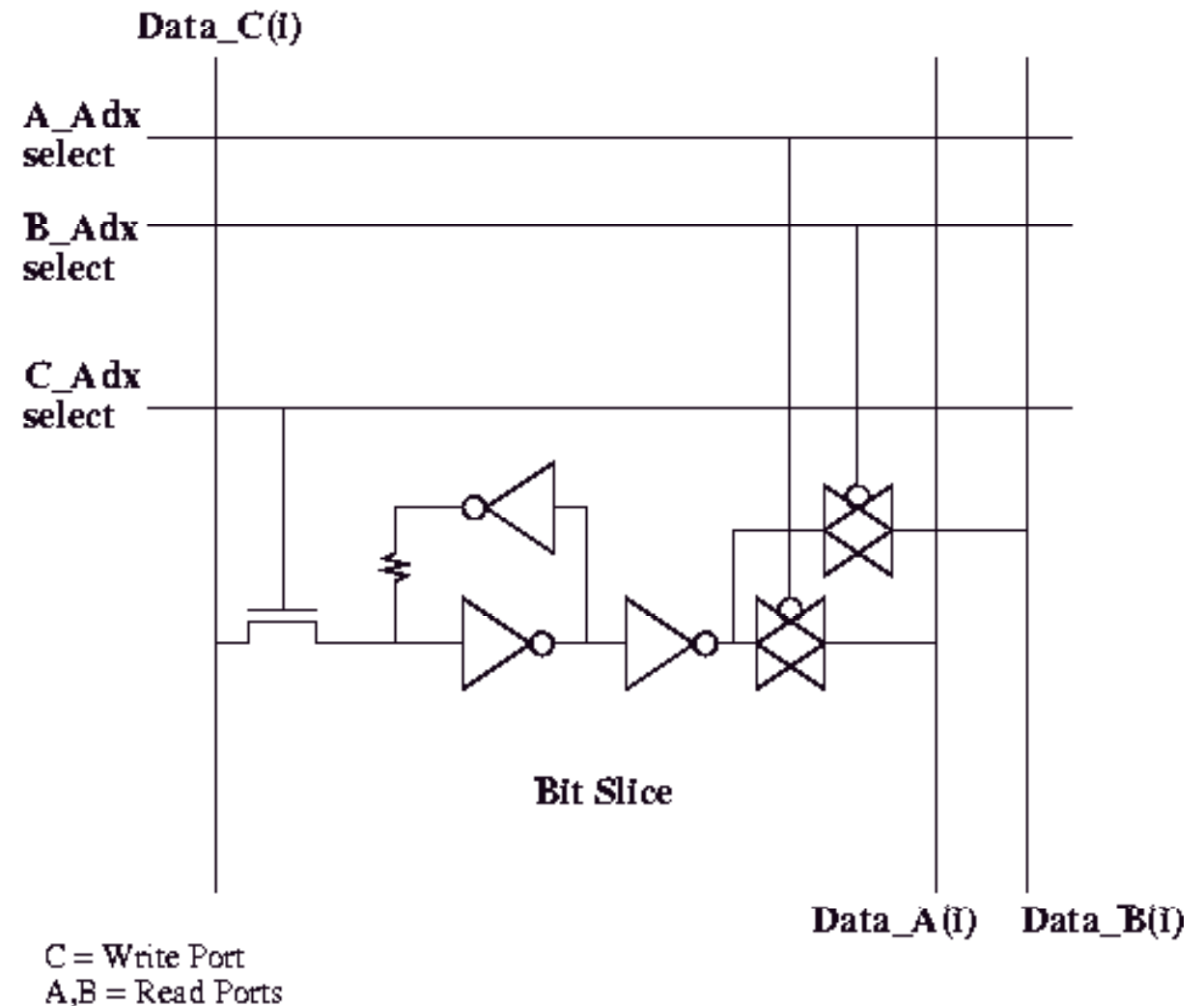
Accès mémoire (attente donnée dispo si store)

Dépendances de donnée mémoire

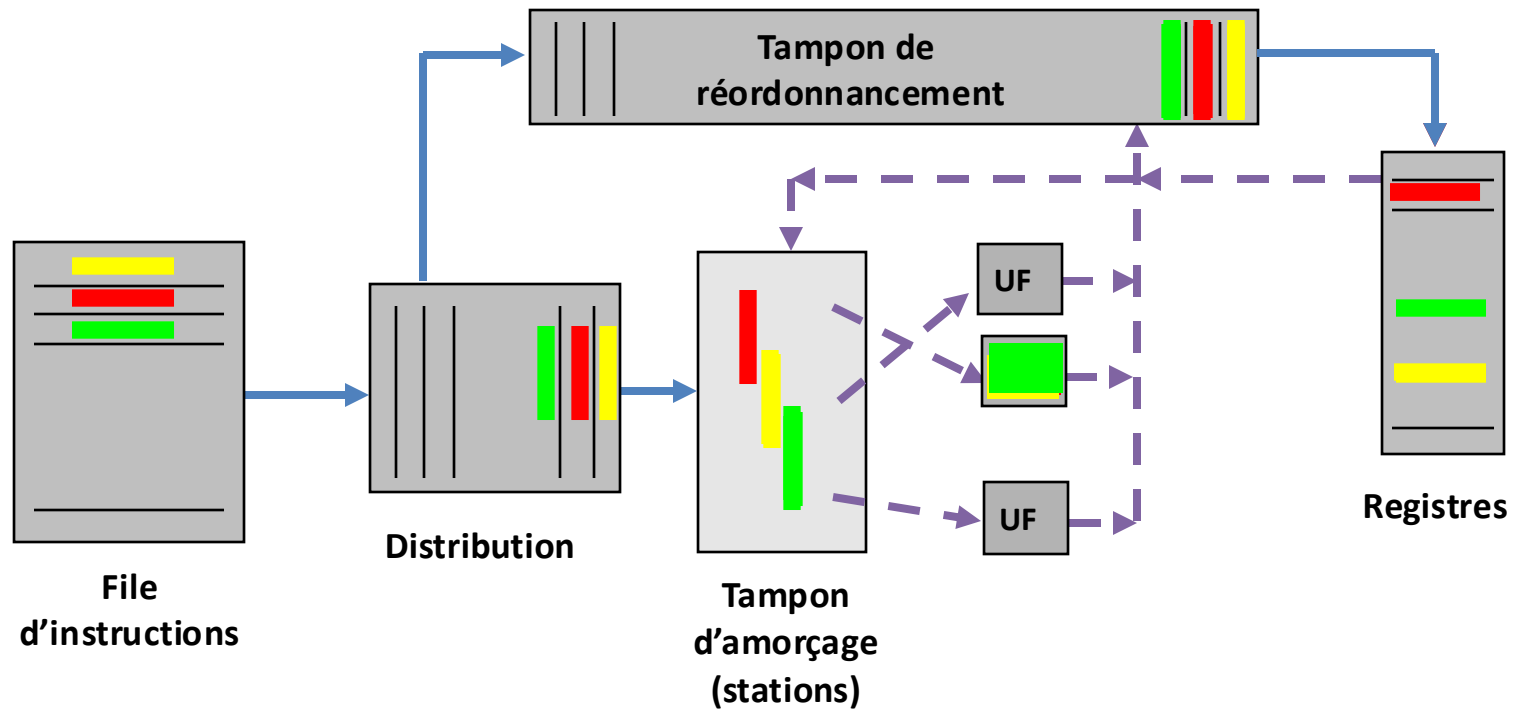
- EAL/EAE : écritures réalisées dans l'ordre
 - Aléas impossibles
- LAE : pour être exécutée dans le désordre, les lectures doivent vérifier les écritures en attente
 - load bypassing, load forwarding
 - La file d'écritures (store queue) garde la trace des écritures en attente
 - adresse + donnée à écrire
 - Les lectures vérifient les adresses dans la file d'écritures
 - L'âge des lectures et des écritures doit être considéré...
- Beaucoup plus complexe qu'il n'y paraît
 - L'âge des lectures et des écritures doit être considéré...
 - Et si l'adresse d'un store n'est pas encore calculée ?
 - Prédiction de dépendance = spéculation = récupération

Augmenter la bande passante

- Augmenter le nombre d'unités traitant les loads et les stores
 - Pas si simple car elles se partagent un état qui est dans le cache
- Eviter de bloquer une unité en cas d'échec dans le cache
 - MSHRs (Miss Status Holding Registers) mémorisent les échecs L1 en cours de résolution (8 dans le pentium4)
 - Réveil des loads bloqués quand échec résolu
- Vraie mémoire multiport
 - En pratique, rarement plus de 2 ports
- Mémoire organisée en bancs
 - Nombre de bancs suffisant pour avoir peu de conflits d'accès à un banc



Maintien de l'état précis

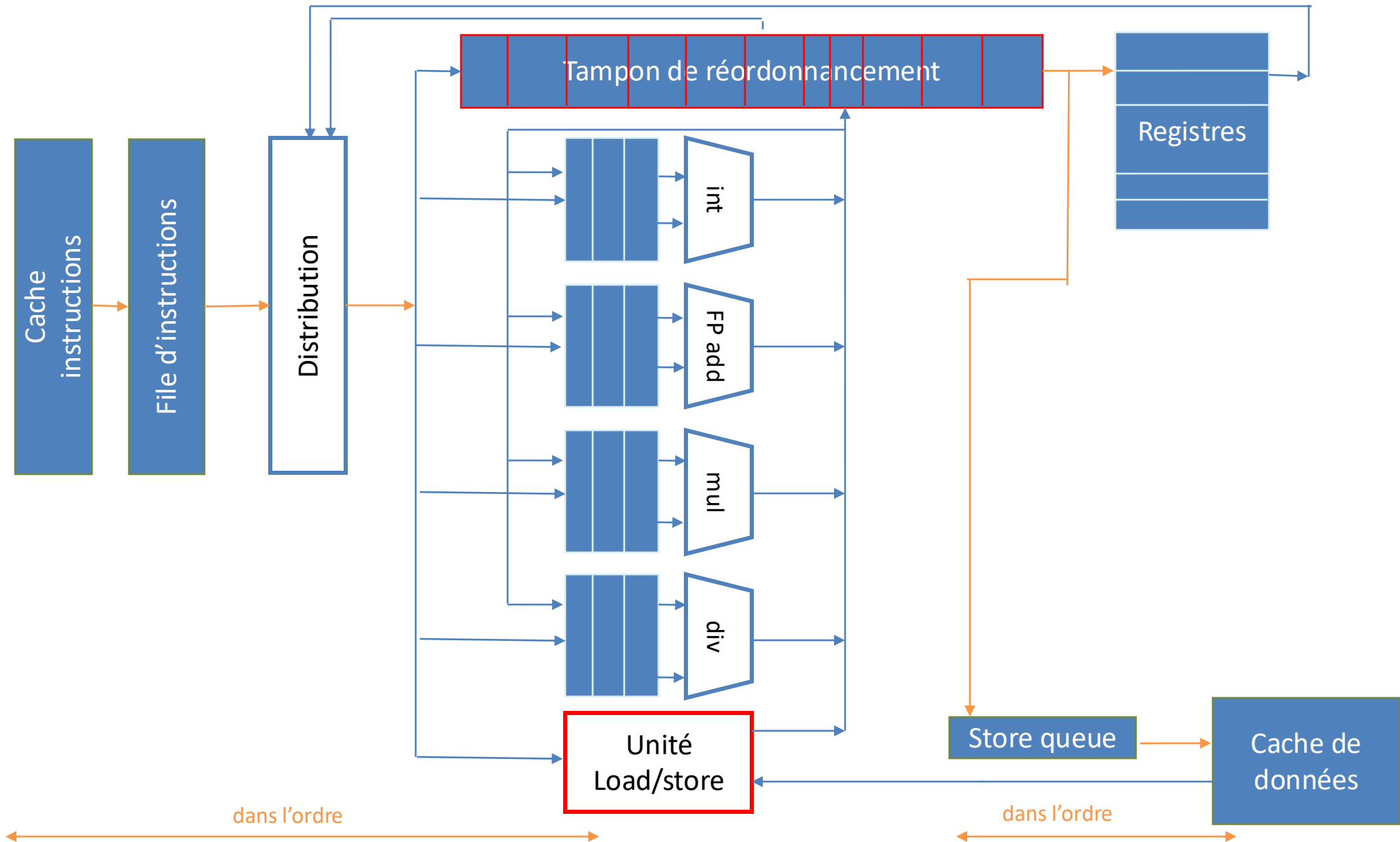


Les instructions rouge et jaune sont indépendantes, la verte dépend de la rouge. Les 3 instructions s'exécutent dans la même UF ☹

Ordonnancement dynamique et spéculatif

- Comment modifier de manière minimale le schéma de Tomasulo pour
 - prendre en compte la spéculation (prédiction de branchement)
 - rendre les exceptions précises

Ordonnancement dynamique et spéculatif



Les structures

Banc de registres

	occupé ?	Qi	data
R0			
R1			
R2			
...			

Une station

Op	Vs1	Vs2	Qs1	Qs2	occupé

Qi, Qs1, Qs2 : numéro de l'entrée dans le ROB qui contient l'instruction

Une entrée du ROB

occupé	n° registre destination	valeur résultat	V	prédiction branch ^t	exception ?	cause except ^o

dest

valeur

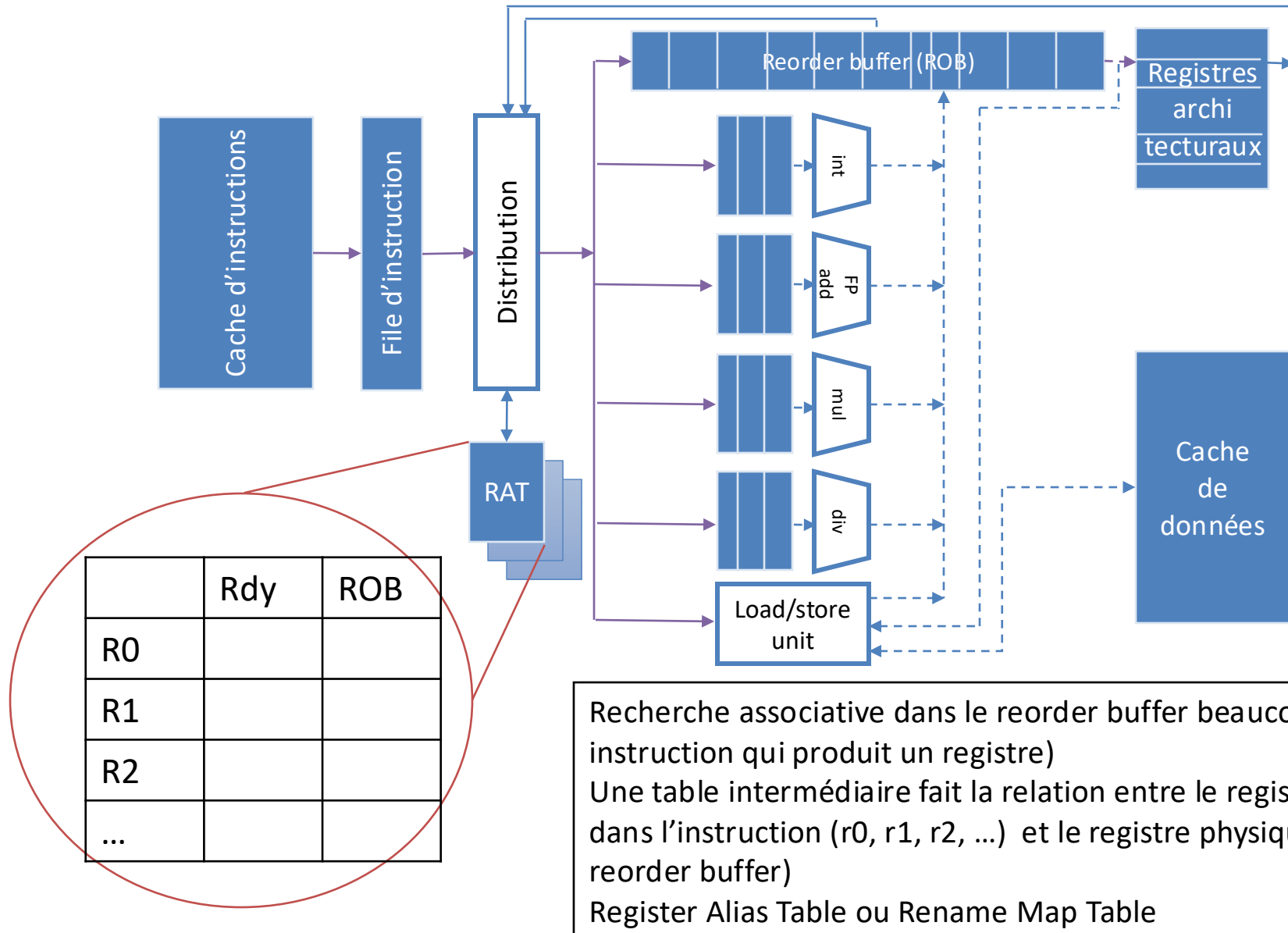
V : résultat prêt

La spéculation matérielle

Les étapes de l'algorithme (non superscalaire)

Etat de l'instruction	Attente jusqu'à	Actions à effectuer
Lancement	Station et ROB disponibles	$\text{Reg}[D].Q_i \leftarrow b$; $\text{ROB}[b].\text{dest} \leftarrow D$; $\text{SR}[r].\text{occupé} \leftarrow \text{oui}$; $\text{SR}[r].\text{dest} \leftarrow b$; si $(\text{Reg}[S1].Q_i == 0)$ alors $\{\text{SR}[r].Vs1 \leftarrow \text{Reg}[S1].\text{data}$; $\text{SR}[r].Qs1 \leftarrow 0\}$ sinon $\text{SR}[r].Qs1 \leftarrow b$ } si $(\text{Reg}[S2].Q_i == 0)$ alors $\{\text{SR}[r].Vs2 \leftarrow \text{Reg}[S2].\text{data}$; $\text{SR}[r].Qs2 \leftarrow 0\}$ sinon $\text{SR}[r].Qs2 \leftarrow b$ }
Exécution	$(\text{SR}[r].Qs1=0)$ et $\text{SR}[r].Qs2=0$	Aucune. Les opérandes sont dans Vs1 et Vs2.
Ecriture résultat	Exécution terminée de r avec numéro du ROB b et bus résultat disponible.	$\forall x, (\text{si } \text{SR}[x].Qs1=b) \text{ alors } \{\text{SR}[x].Vs1 \leftarrow \text{résultat}$; $\text{SR}[x].Qs1 \leftarrow 0$;} $\forall x, (\text{si } \text{SR}[x].Qs2=b) \text{ alors } \{\text{SR}[x].Vs2 \leftarrow \text{résultat}$; $\text{SR}[x].Qs2 \leftarrow 0$;} $\text{SR}[r].\text{occupé} \leftarrow \text{non}$; $\text{ROB}[b].\text{valeur} \leftarrow \text{résultat}$;
Garantie	Instruction en tête du ROB (entrée t) et écriture du résultat effectuée ($V=1$).	si $(\text{ROB}[t].\text{instruction} = \text{branchement})$ et (branchement mal prédit) alors {vider le ROB ; redémarrer la lecture des instructions} sinon { si $\text{ROB}[t].V=\text{oui}$ { si $(\text{ROB}[t].\text{instruction} = \text{rangement})$ alors $\{\text{Mem}(\text{ROB}[t].\text{valeur}) \leftarrow \text{ROB}[t].\text{valeur}$;} sinon $\text{Reg}[\text{ROB}[t].\text{dest}].\text{data} \leftarrow \text{ROB}[t].\text{valeur}$; $\text{Reg}[\text{ROB}[t].\text{dest}].Q_i \leftarrow 0$;}}

Séparation banc de registre et champs de renommage

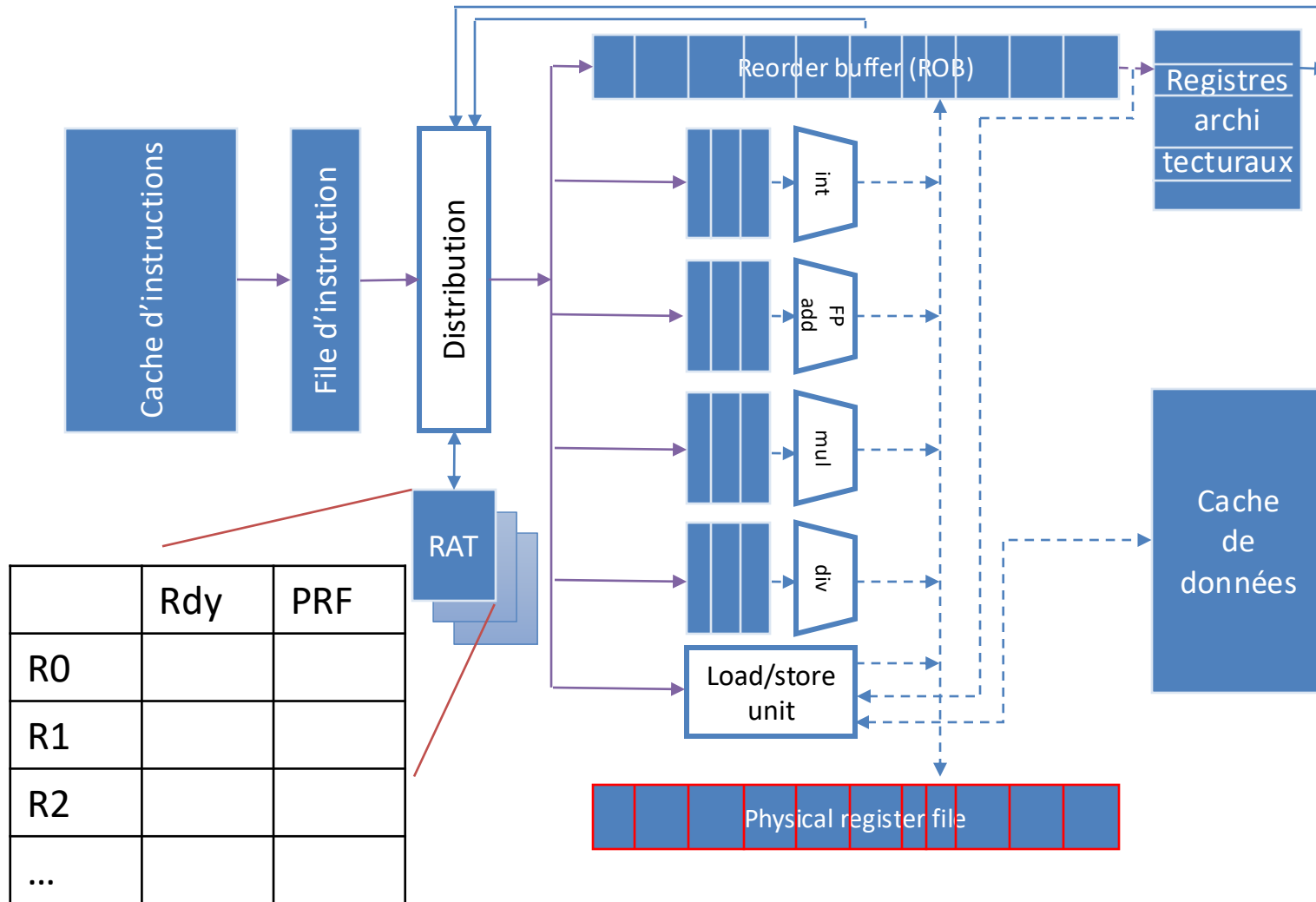


Recherche associative dans le reorder buffer beaucoup trop « coûteuse » (dernière instruction qui produit un registre)

Une table intermédiaire fait la relation entre le registre architectural mentionné dans l'instruction (r0, r1, r2, ...) et le registre physique (numéro d'entrée dans le reorder buffer)

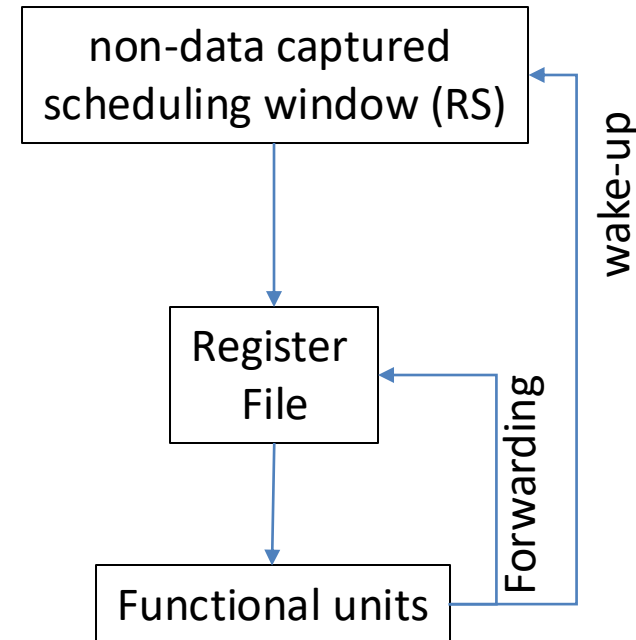
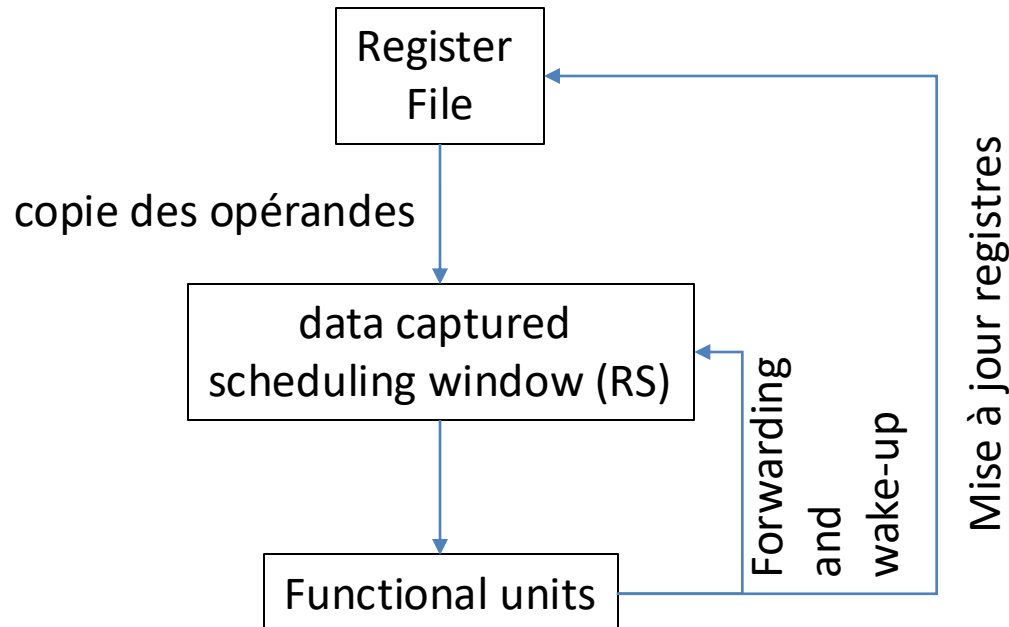
Register Alias Table ou Rename Map Table

Banc de registres physiques



Lecture des opérandes

- Tomasulo capture les valeurs (si elles sont prêtes) lors de la distribution (dispatch) pour les recopier dans une SR. Les valeurs sont donc présentes dans le ROB (ou le PRF ou l'ARF) **et** dans la SR.
- Possibilité de ne lire les valeurs (dans le ROB ou le PRF ou l'ARF) lors de la mise à feu (issue). Les valeurs ne sont pas recopiées dans les SR, seules les informations de disponibilités des valeurs sont présentes.



- Bref, il y a énormément d'options lorsqu'on veut réaliser une machine aussi complexe !